

01 — Action Potentials and the Hodgkin-Huxley Model

BMED365 – Computational Medicine (Lab 5)

Arvid Lundervold w/ Claude (Anthropic) & Grok (xAI) — 2025-02-22

Updated: 2026-02-19 (Cursor 2.5.17 with Claude Opus 4.6)

Updated: 2026-02-20 (Cursor 2.5.20 with Claude Opus 4.6) - HHM Simulator

1. Motivation & Learning Objectives

Why study the action potential?

The **action potential** is the fundamental unit of communication in the nervous system. Every thought you think, every movement you make, every sensation you perceive — all depend on the rapid, precisely orchestrated electrical impulses that travel along neurons. Understanding the action potential is therefore central to understanding the brain itself.

In 1952, Alan Hodgkin and Andrew Huxley published a series of landmark papers that provided the first quantitative, biophysical model of the action potential. Their work on the **squid giant axon** — a remarkably large nerve fiber ideal for electrophysiological experiments — resulted in a set of nonlinear ordinary differential equations that accurately reproduce the shape, timing, and threshold behavior of real action potentials.

The **Hodgkin-Huxley (HH) model** is widely considered one of the greatest achievements in mathematical biology. It earned Hodgkin and Huxley the **Nobel Prize in Physiology or Medicine in 1963** and laid the groundwork for the entire field of **computational neuroscience**.

What you will learn

In this notebook you will:

1. **Understand** the biological basis of the action potential — what happens at the level of ion channels in the cell membrane
2. **Appreciate** the historical context — how voltage-clamp experiments on squid giant axons led to a mathematical revolution

3. **Master** the mathematical formulation of the Hodgkin-Huxley model — the equivalent circuit, the coupled ODEs, and the gating kinetics
4. **Implement** the model in Python and simulate action potentials numerically
5. **Visualize** and interpret the dynamics of membrane potential, gating variables, and ionic currents
6. **Explore** clinical and scientific implications — from anesthetics to cardiac arrhythmias to neural prosthetics

2. Historical Context: The Squid, the Scientists, and the Nobel Prize

The quest to understand nerve impulses

The electrical nature of nerve signals was suspected since Luigi Galvani's experiments in the 1780s, but it took nearly two centuries to develop a quantitative understanding. Key milestones include:

Year	Milestone	Scientists
1791	Discovery of "animal electricity"	Luigi Galvani
1850s	Measurement of nerve impulse velocity (~30 m/s)	Hermann von Helmholtz
1902	Membrane theory of nerve excitation	Julius Bernstein
1939	First intracellular recording from squid giant axon	Hodgkin & Huxley
1949	Development of voltage-clamp technique	Cole & Marmont; Hodgkin, Huxley & Katz
1952	Publication of the Hodgkin-Huxley model	Hodgkin & Huxley
1963	Nobel Prize in Physiology or Medicine	Hodgkin, Huxley & Eccles

Why the squid?

The **giant axon** of the North Atlantic squid (*Loligo*) is up to **1 mm in diameter** — about 100 to 1000 times thicker than a typical mammalian axon. This enormous size made it possible to:

- Insert electrodes *inside* the axon
- Measure membrane potential directly
- Apply the **voltage-clamp technique** to control membrane voltage and measure ionic currents independently

 Squid giant axon research

Researcher examining giant squid axon fibers at Woods Hole, Massachusetts. NIH researchers such as Dr. Kenneth Cole and Dr. Tasaki conducted pioneering work on axon electrophysiology here. Source: NIH History Office / Wikimedia Commons (public domain).

The voltage-clamp revolution

The key experimental innovation was the **voltage clamp**: a feedback amplifier circuit that holds the membrane potential at a chosen value (the "command voltage") while measuring the current that flows through the membrane. By systematically varying the command voltage and using ion-substitution experiments, Hodgkin and Huxley were able to:

1. **Separate** the total membrane current into sodium, potassium, and leak components
2. **Characterize** the voltage- and time-dependence of each ionic conductance
3. **Fit** mathematical expressions to the experimental data
4. **Reconstruct** the action potential by solving the resulting differential equations — by hand, using a mechanical calculator!

"We were lucky in having a preparation [the squid giant axon] with which we could do experiments that were not possible with other nerve fibres." —

Alan Hodgkin

The 1952 papers

Hodgkin and Huxley published five consecutive papers in the *Journal of Physiology* in 1952:

1. Measurement of membrane currents (H&H, 1952a)
2. The components of membrane conductance (H&H, 1952b)
3. Sodium and potassium conductance changes (H&H, 1952c)
4. The inactivation process (H&H, 1952d)
5. **A quantitative description of membrane current and its application to conduction and excitation in nerve** (H&H, 1952e) — the full model

The fifth paper presented the complete mathematical model and showed that it could reproduce: the shape of the action potential, its threshold, the refractory period, propagation velocity, repetitive firing, and many other experimentally observed phenomena.

```
In [29]: # --- Colab setup ---
import sys
if 'google.colab' in sys.modules:
    %matplotlib inline
    import urllib.request, os
    os.makedirs('../assets', exist_ok=True)
```

```

_base = 'https://raw.githubusercontent.com/arvidl/BMED365-2026/main/Lab5
for f in ['squid-giant-axon.jpg', 'voltage-clamp.png']:
    if not os.path.exists(f'../assets/{f}'):
        urllib.request.urlretrieve(_base + f, f'../assets/{f}')
else:
    %matplotlib inline

```

```

In [30]: # --- Colab setup ---
import sys
if 'google.colab' in sys.modules:
    %matplotlib inline
    import urllib.request, os
    os.makedirs('../assets', exist_ok=True)
    _base = 'https://raw.githubusercontent.com/arvidl/BMED365-2026/main/Lab5
    for f in ['squid-giant-axon.jpg', 'voltage-clamp.png']:
        if not os.path.exists(f'../assets/{f}'):
            urllib.request.urlretrieve(_base + f, f'../assets/{f}')
else:
    %matplotlib inline

```

```

In [31]: from IPython.display import Image, display, HTML

# Display a schematic of the voltage-clamp setup
display(HTML('''
<figure style="text-align:center; margin: 20px 0;">

<figcaption style="font-size:0.9em; color:#555; margin-top:8px;">
<b>Figure 1.</b> Schematic of the voltage-clamp technique. A feedback amplif
the membrane potential at the command voltage, while a separate electrode me
<br/><i>Source: Wikimedia Commons (CC BY-SA 3.0).</i>
</figcaption>
</figure>
'''))

```



Voltage-clamp schematic

Figure 1. Schematic of the voltage-clamp technique. A feedback amplifier injects current to keep the membrane potential at the command voltage, while a separate electrode measures the required current.

Source: Wikimedia Commons (CC BY-SA 3.0).

3. The Action Potential: Biological Background

What is an action potential?

An **action potential** (AP) is a brief, all-or-nothing reversal of the electrical potential across a neuron's cell membrane. At rest, the inside of the neuron is about **−65 to −70 mV** relative to the outside (the **resting membrane potential**). During an action

potential, the membrane potential rapidly swings to about **+30 to +40 mV** and then returns to rest — all within about **1–2 milliseconds**.

The five phases

The action potential can be divided into five distinct phases:

Phase	Description	Key mechanism
1. Resting state	Membrane at ~ -65 mV	Na^+ and K^+ channels mostly closed; leak channels maintain resting potential
2. Depolarization	Rapid rise to $\sim +40$ mV	Voltage-gated Na^+ channels open $\rightarrow$ Na^+ rushes in
3. Peak / Overshoot	Membrane potential peaks	Na^+ channels begin to inactivate
4. Repolarization	Return toward resting potential	Na^+ channels inactivate; K^+ channels open $\rightarrow$ K^+ flows out
5. Hyperpolarization	Brief undershoot below rest	K^+ channels still open; membrane temporarily more negative than rest

Ion channels: the molecular machinery

The action potential depends on two main types of **voltage-gated ion channels** embedded in the cell membrane:

- **Voltage-gated Na^+ channels:** Have two gates — an *activation gate* (fast, controlled by gating variable m) and an *inactivation gate* (slow, controlled by h). The channel conducts only when both gates are open.
- **Voltage-gated K^+ channels:** Have a single *activation gate* (controlled by n), which opens more slowly than the Na^+ activation gate.
- **Leak channels:** Always open, providing a small background conductance that helps maintain the resting potential.

Check Your Understanding: The Action Potential

Q1. During which phase of the action potential is the membrane potential *positive* (inside more positive than outside)?

► Click to reveal answer

Q2. Why can't the membrane potential reach the Na^+ equilibrium potential ($E_{\text{Na}} = +50$ mV) during an action potential, even though Na^+ channels are open?

► Click to reveal answer

Q3. A patient receives a local anesthetic (e.g., lidocaine) at the dentist. Based on what you know about action potentials, what molecular target does this drug act on, and why does it block pain?

► Click to reveal answer

```
In [32]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
np.random.seed(42)

# --- Programmatic illustration of an idealized action potential ---
fig, ax = plt.subplots(figsize=(11, 6))

# Create an idealized action potential shape using piecewise functions
t_ap = np.array([0, 5, 6, 6.3, 6.6, 6.8, 7.3, 8.0, 8.5, 9.5, 11, 15])
v_ap = np.array([-65, -65, -55, -20, 20, 40, 0, -70, -75, -68, -65, -65])

# Interpolate for smooth curve
from scipy.interpolate import PchipInterpolator
t_fine = np.linspace(0, 15, 500)
interp = PchipInterpolator(t_ap, v_ap)
v_fine = interp(t_fine)

ax.plot(t_fine, v_fine, 'k-', linewidth=2.5)

# Threshold line
ax.axhline(y=-55, color='gray', linestyle='--', linewidth=0.8, alpha=0.7)
ax.text(0.3, -52, 'Threshold (~-55 mV)', fontsize=9, color='gray')

# Resting potential line
ax.axhline(y=-65, color='gray', linestyle=':', linewidth=0.8, alpha=0.5)
ax.text(0.3, -62, 'Resting potential (~-65 mV)', fontsize=9, color='gray')

# Phase annotations with colored regions
phases = [
    (0, 5.8, -80, 50, '#e8f4f8', '①\nResting\nstate'),
    (5.8, 6.9, -80, 50, '#ffe6e6', '②\nDepolar-\nization'),
    (6.9, 7.2, -80, 50, '#fff3e0', '③\nPeak'),
    (7.2, 8.3, -80, 50, '#e8f5e9', '④\nRepolar-\nization'),
    (8.3, 10.0, -80, 50, '#f3e5f5', '⑤\nHyper-\npolarization'),
]

for t0, t1, ymin, ymax, color, label in phases:
    ax.axvspan(t0, t1, alpha=0.3, color=color)
    ax.text((t0 + t1) / 2, 45, label, ha='center', va='top', fontsize=8, for

# Arrow annotations for Na+ and K+
ax.annotate('Na+ influx\n(channels open)', xy=(6.5, 10), xytext=(3.5, 35),
            fontsize=9, color='red', fontweight='bold',
            arrowprops=dict(arrowstyle='->', color='red', lw=1.5))

ax.annotate('K+ efflux\n(channels open)', xy=(7.8, -40), xytext=(10, -20),
            fontsize=9, color='blue', fontweight='bold',
            arrowprops=dict(arrowstyle='->', color='blue', lw=1.5))
```

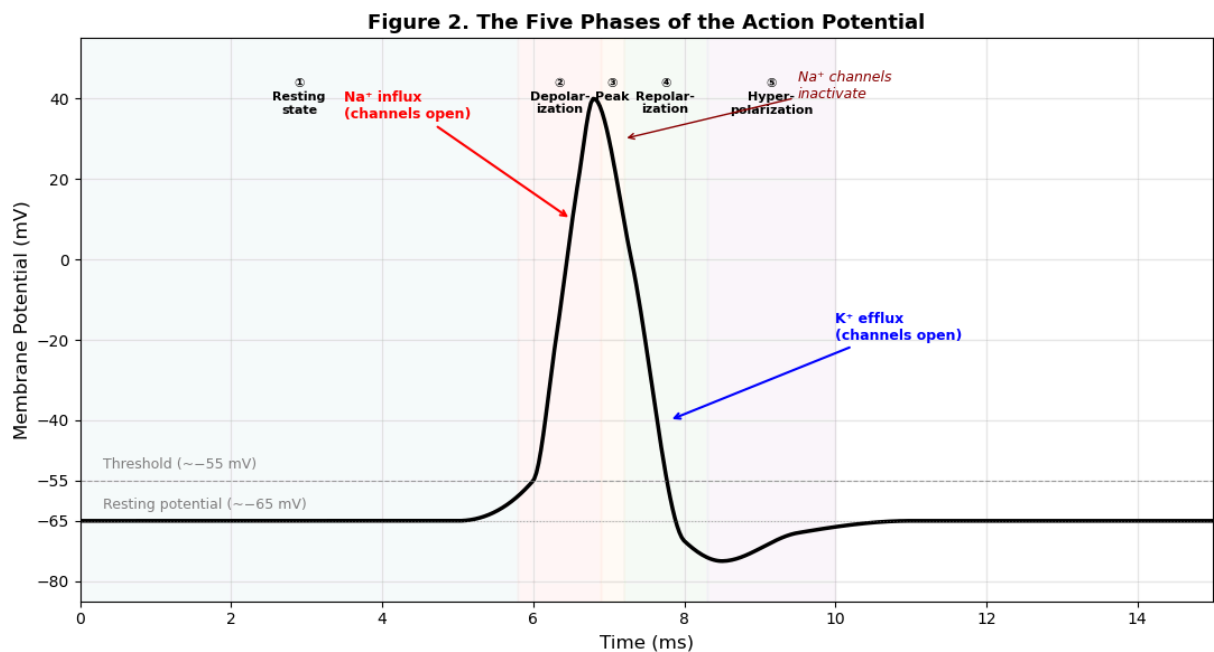
```

ax.annotate('Na+ channels\ninactivate', xy=(7.2, 30), xytext=(9.5, 40),
           fontsize=9, color='darkred', style='italic',
           arrowprops=dict(arrowstyle='->', color='darkred', lw=1.0))

ax.set_xlabel('Time (ms)', fontsize=12)
ax.set_ylabel('Membrane Potential (mV)', fontsize=12)
ax.set_title('Figure 2. The Five Phases of the Action Potential', fontsize=12)
ax.set_xlim(0, 15)
ax.set_ylim(-85, 55)
ax.set_yticks([-80, -65, -55, -40, -20, 0, 20, 40])
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



4. Scientific and Clinical Implications

Understanding the action potential and the Hodgkin-Huxley model has had far-reaching consequences in both basic science and clinical medicine:

Neuroscience & Brain Function

- The HH model is the foundation of **computational neuroscience**. Modern large-scale brain simulations (e.g., the Blue Brain Project, the Human Brain Project) use HH-type models for individual neurons
- Understanding how networks of neurons generate complex behavior (perception, memory, motor control) begins with understanding the single-neuron action potential

Pharmacology & Anesthetics

- **Local anesthetics** (e.g., lidocaine) work by blocking voltage-gated Na^+ channels, preventing action potential propagation in pain-sensing neurons
- **Antiepileptic drugs** (e.g., carbamazepine, phenytoin) modulate Na^+ or K^+ channel kinetics to reduce excessive neuronal firing
- The HH model provides a quantitative framework for predicting how channel-blocking drugs will affect neural excitability

Cardiac Electrophysiology

- The heart's electrical conduction system uses action potentials similar in principle (though different in detail) to those in neurons
- **Cardiac arrhythmias** can result from dysfunctional ion channels (channelopathies), and the HH framework has been extended to model cardiac cells (e.g., the Luo-Rudy, O'Hara-Rudy models)
- **Anti-arrhythmic drugs** are classified by which ion channels they target (the Vaughan-Williams classification)

Neurological Disorders

- **Multiple sclerosis** — demyelination disrupts action potential propagation along axons
- **Epilepsy** — excessive, synchronized neuronal firing traced to ion channel dysfunction
- **Channelopathies** — genetic mutations in Na^+ , K^+ , or Ca^{2+} channels cause conditions like episodic ataxia, long QT syndrome, and familial hemiplegic migraine

Neural Engineering & Prosthetics

- **Deep brain stimulation** for Parkinson's disease relies on understanding how electrical stimulation modulates action potentials
- **Cochlear implants** and **retinal prosthetics** must interface with neurons at the level of action potential generation
- **Brain-computer interfaces** record and decode patterns of action potentials

The Hodgkin-Huxley model remains, after more than 70 years, one of the most successful quantitative models in all of biology.

5. The Hodgkin-Huxley Model: Conceptual Framework

The membrane as an electrical circuit

Hodgkin and Huxley's key insight was to model the cell membrane as an **equivalent electrical circuit**:

- The **lipid bilayer** acts as a **capacitor** (C_m), storing charge across the membrane
- Each type of **ion channel** acts as a **variable conductance** (resistor) in parallel with the capacitor
- Each ionic conductance is in series with a **battery** (the Nernst/reversal potential for that ion)
- **External current** (I_{ext}) can be injected to stimulate the neuron

The total membrane current is the sum of capacitive current and ionic currents:

$$I_{\text{total}} = C_m \frac{dV}{dt} + I_{\text{ionic}}$$

This leads to the central equation of the HH model:

$$C_m \frac{dV}{dt} = I_{\text{ext}} - \underbrace{g_{\text{Na}} m^3 h (V - E_{\text{Na}})}_{I_{\text{Na}}} - \underbrace{g_{\text{K}} n^4 (V - E_{\text{K}})}_{I_{\text{K}}} - \underbrace{g_{\text{L}} (V - E_{\text{L}})}_{I_{\text{L}}}$$

```
In [33]: import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch

fig, ax = plt.subplots(figsize=(12, 7))
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)
ax.set_aspect('equal')
ax.axis('off')

# --- Draw the equivalent circuit ---

# Membrane capacitor
cap_x, cap_y = 2.5, 4.0
ax.plot([cap_x, cap_x], [cap_y + 0.8, cap_y + 0.3], 'k-', lw=2)
ax.plot([cap_x - 0.4, cap_x + 0.4], [cap_y + 0.3, cap_y + 0.3], 'k-', lw=3)
ax.plot([cap_x - 0.4, cap_x + 0.4], [cap_y - 0.1, cap_y - 0.1], 'k-', lw=3)
ax.plot([cap_x, cap_x], [cap_y - 0.1, cap_y - 0.6], 'k-', lw=2)
ax.text(cap_x + 0.6, cap_y + 0.1, r'$C_m$', fontsize=14, fontweight='bold')

# Na channel (variable resistor + battery)
na_x = 5.0
# Variable resistor (zigzag approx. as box)
rect_na = FancyBboxPatch((na_x - 0.35, cap_y - 0.2), 0.7, 0.6,
                          boxstyle="round,pad=0.05", facecolor='#ffcccc',
                          edgecolor='red', linewidth=2)
ax.add_patch(rect_na)
ax.text(na_x, cap_y + 0.1, r'$g_{\text{Na}} m^3 h$', fontsize=10, ha='center', va='center',
        color='red', fontweight='bold')

# Battery
ax.plot([na_x, na_x], [cap_y - 0.2, cap_y - 0.7], 'r-', lw=2)
```

```

ax.plot([na_x - 0.3, na_x + 0.3], [cap_y - 0.7, cap_y - 0.7], 'r-', lw=3)
ax.plot([na_x - 0.15, na_x + 0.15], [cap_y - 0.9, cap_y - 0.9], 'r-', lw=2)
ax.plot([na_x, na_x], [cap_y - 0.9, cap_y - 1.2], 'r-', lw=2)
ax.text(na_x + 0.55, cap_y - 0.8, r'$E_{Na}$', fontsize=11, color='red')
# Wires
ax.plot([na_x, na_x], [cap_y + 0.6, cap_y + 0.8], 'k-', lw=2)
ax.plot([na_x, na_x], [cap_y - 1.2, cap_y - 1.4], 'k-', lw=2)
ax.text(na_x, cap_y + 1.1, 'Na+', fontsize=12, ha='center', color='red', for

# K channel (variable resistor + battery)
k_x = 7.5
rect_k = FancyBboxPatch((k_x - 0.35, cap_y - 0.2), 0.7, 0.6,
                        boxstyle="round,pad=0.05", facecolor='#cce5ff',
                        edgecolor='blue', linewidth=2)
ax.add_patch(rect_k)
ax.text(k_x, cap_y + 0.1, r'$g_K n^4$', fontsize=10, ha='center', va='center',
        color='blue', fontweight='bold')
ax.plot([k_x, k_x], [cap_y - 0.2, cap_y - 0.7], 'b-', lw=2)
ax.plot([k_x - 0.3, k_x + 0.3], [cap_y - 0.7, cap_y - 0.7], 'b-', lw=3)
ax.plot([k_x - 0.15, k_x + 0.15], [cap_y - 0.9, cap_y - 0.9], 'b-', lw=2)
ax.plot([k_x, k_x], [cap_y - 0.9, cap_y - 1.2], 'b-', lw=2)
ax.text(k_x + 0.55, cap_y - 0.8, r'$E_K$', fontsize=11, color='blue')
ax.plot([k_x, k_x], [cap_y + 0.6, cap_y + 0.8], 'k-', lw=2)
ax.plot([k_x, k_x], [cap_y - 1.2, cap_y - 1.4], 'k-', lw=2)
ax.text(k_x, cap_y + 1.1, 'K+', fontsize=12, ha='center', color='blue', font

# Leak channel (fixed resistor + battery)
l_x = 10.0
rect_l = FancyBboxPatch((l_x - 0.35, cap_y - 0.2), 0.7, 0.6,
                        boxstyle="round,pad=0.05", facecolor='#e0f0e0',
                        edgecolor='green', linewidth=2)
ax.add_patch(rect_l)
ax.text(l_x, cap_y + 0.1, r'$g_L$', fontsize=10, ha='center', va='center',
        color='green', fontweight='bold')
ax.plot([l_x, l_x], [cap_y - 0.2, cap_y - 0.7], 'g-', lw=2)
ax.plot([l_x - 0.3, l_x + 0.3], [cap_y - 0.7, cap_y - 0.7], 'g-', lw=3)
ax.plot([l_x - 0.15, l_x + 0.15], [cap_y - 0.9, cap_y - 0.9], 'g-', lw=2)
ax.plot([l_x, l_x], [cap_y - 0.9, cap_y - 1.2], 'g-', lw=2)
ax.text(l_x + 0.55, cap_y - 0.8, r'$E_L$', fontsize=11, color='green')
ax.plot([l_x, l_x], [cap_y + 0.6, cap_y + 0.8], 'k-', lw=2)
ax.plot([l_x, l_x], [cap_y - 1.2, cap_y - 1.4], 'k-', lw=2)
ax.text(l_x, cap_y + 1.1, 'Leak', fontsize=12, ha='center', color='green', f

# Top wire (extracellular)
ax.plot([cap_x, l_x], [cap_y + 0.8, cap_y + 0.8], 'k-', lw=2)
ax.text(6.0, cap_y + 1.6, 'EXTRACELLULAR (outside)', fontsize=12, ha='center',
        fontweight='bold', style='italic', color='#444')

# Bottom wire (intracellular)
ax.plot([cap_x, l_x], [cap_y - 1.4, cap_y - 1.4], 'k-', lw=2)
ax.text(6.0, cap_y - 2.1, 'INTRACELLULAR (inside)', fontsize=12, ha='center',
        fontweight='bold', style='italic', color='#444')

# Membrane box
membrane_rect = FancyBboxPatch((1.5, cap_y - 2.5), 9.5, 4.6,
                                boxstyle="round,pad=0.2", facecolor='none',

```

```

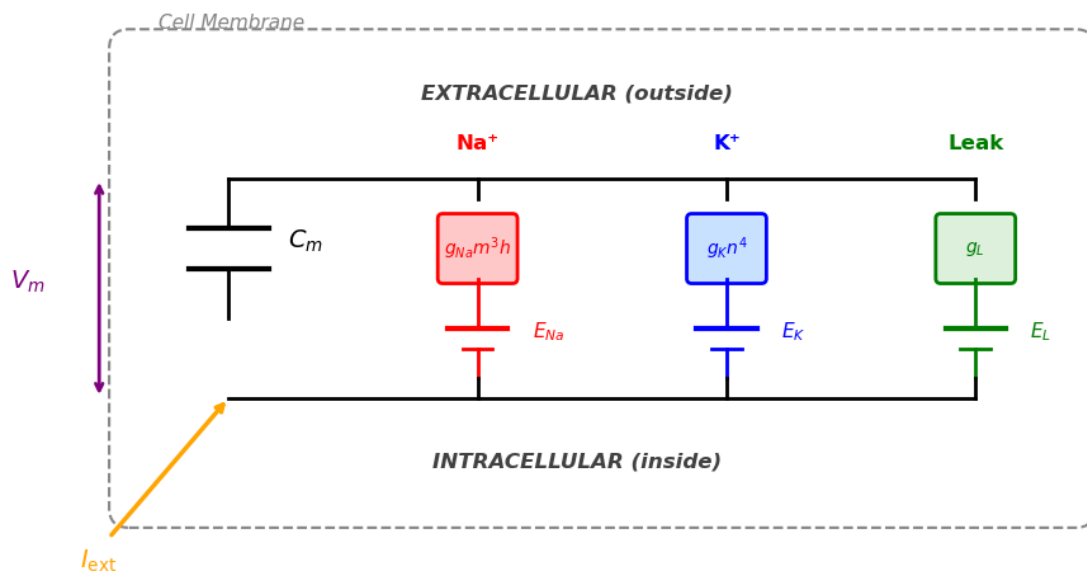
edgecolor='#888', linewidth=1.5, linestyle='
ax.add_patch(membrane_rect)
ax.text(1.8, cap_y + 2.3, 'Cell Membrane', fontsize=11, color='#888', style='
# External current arrow
ax.annotate('', xy=(cap_x, cap_y - 1.4), xytext=(cap_x - 1.2, cap_y - 2.8),
            arrowprops=dict(arrowstyle='->', color='orange', lw=2.5))
ax.text(cap_x - 1.5, cap_y - 3.1, r'$I_{\mathrm{ext}}$', fontsize=14, color='c
        fontweight='bold')

# Voltage label
ax.annotate('', xy=(1.2, cap_y + 0.8), xytext=(1.2, cap_y - 1.4),
            arrowprops=dict(arrowstyle='<-->', color='purple', lw=2))
ax.text(0.3, cap_y - 0.3, r'$V_m$', fontsize=14, color='purple', fontweight='
ax.set_title('Figure 3. Equivalent Electrical Circuit of the Hodgkin-Huxley
            fontsize=13, fontweight='bold', pad=20)

plt.tight_layout()
plt.show()

```

Figure 3. Equivalent Electrical Circuit of the Hodgkin-Huxley Model



6. Mathematical Formulation

The coupled ODE system

The Hodgkin-Huxley model consists of **four coupled ordinary differential equations** — one for the membrane potential V and three for the gating variables m , h , n :

Membrane potential:

$$C_m \frac{dV}{dt} = I_{\text{ext}} - g_{\text{Na}} m^3 h (V - E_{\text{Na}}) - g_{\text{K}} n^4 (V - E_{\text{K}}) - g_{\text{L}} (V - E_{\text{L}})$$

Gating variables:

Each gating variable $x \in \{m, h, n\}$ follows first-order kinetics:

$$\frac{dx}{dt} = \alpha_x(V) (1 - x) - \beta_x(V) x$$

This can also be written in terms of steady-state activation $x_{\infty}(V)$ and time constant $\tau_x(V)$:

$$\frac{dx}{dt} = \frac{x_{\infty}(V) - x}{\tau_x(V)}$$

where $x_{\infty}(V) = \frac{\alpha_x}{\alpha_x + \beta_x}$ and $\tau_x(V) = \frac{1}{\alpha_x + \beta_x}$.

Voltage-dependent rate constants

The rate constants α and β for each gating variable are empirical functions fit to squid axon data:

Variable	$\alpha_x(V)$	$\beta_x(V)$
m (Na ⁺ activation)	$\frac{0.1 (V+40)}{1 - e^{-(V+40)/10}}$	$4 e^{-(V+65)/18}$
h (Na ⁺ inactivation)	$0.07 e^{-(V+65)/20}$	$\frac{1}{1 + e^{-(V+35)/10}}$
n (K ⁺ activation)	$\frac{0.01 (V+55)}{1 - e^{-(V+55)/10}}$	$0.125 e^{-(V+65)/80}$

Standard parameter values

Parameter	Symbol	Value	Unit
Membrane capacitance	C_m	1.0	$\mu\text{F}/\text{cm}^2$
Na ⁺ max. conductance	g_{Na}	120.0	mS/cm^2
K ⁺ max. conductance	g_{K}	36.0	mS/cm^2
Leak conductance	g_{L}	0.3	mS/cm^2
Na ⁺ reversal potential	E_{Na}	+50.0	mV
K ⁺ reversal potential	E_{K}	-77.0	mV
Leak reversal potential	E_{L}	-54.4	mV

Understanding the gating exponents

Why m^3h and n^4 ? Hodgkin and Huxley interpreted these as reflecting **independent gating particles**:

- The Na⁺ channel requires **3 activation particles** (m) to be in the "open" position AND **1 inactivation particle** (h) to *not* be blocking → conductance $\propto m^3h$
- The K⁺ channel requires **4 activation particles** (n) → conductance $\propto n^4$

Modern molecular biology has largely confirmed this picture: voltage-gated Na⁺ channels have 4 homologous domains (3 contribute to activation, 1 has the inactivation "ball-and-chain"), and K⁺ channels are tetramers of identical subunits.

Check Your Understanding: The Mathematics

Q4. In the HH model, why does the Na⁺ current use m^3h while the K⁺ current uses n^4 ? What is the physical interpretation of these exponents?

► Click to reveal answer

Q5. What happens to the gating variable m if the membrane is suddenly stepped to a very depolarized voltage (e.g., +20 mV) and held there? Describe qualitatively what α_m and β_m do.

► Click to reveal answer

Q6. (DIY) Using the steady-state formulas, calculate m_∞ , h_∞ , and n_∞ at $V = 0$ mV. Which channels are mostly open? Which are mostly closed?

► Click for hint

► Click to reveal answer

```
In [34]: import numpy as np
import matplotlib.pyplot as plt

# --- Voltage-dependent rate constants and steady-state curves ---

V = np.linspace(-100, 50, 500)

# Rate constants
def alpha_m(V): return 0.1 * (V + 40) / (1 - np.exp(-(V + 40) / 10))
def beta_m(V): return 4.0 * np.exp(-(V + 65) / 18)
def alpha_h(V): return 0.07 * np.exp(-(V + 65) / 20)
def beta_h(V): return 1.0 / (1 + np.exp(-(V + 35) / 10))
def alpha_n(V): return 0.01 * (V + 55) / (1 - np.exp(-(V + 55) / 10))
def beta_n(V): return 0.125 * np.exp(-(V + 65) / 80)

# Steady-state values
```

```

m_inf = alpha_m(V) / (alpha_m(V) + beta_m(V))
h_inf = alpha_h(V) / (alpha_h(V) + beta_h(V))
n_inf = alpha_n(V) / (alpha_n(V) + beta_n(V))

# Time constants
tau_m = 1.0 / (alpha_m(V) + beta_m(V))
tau_h = 1.0 / (alpha_h(V) + beta_h(V))
tau_n = 1.0 / (alpha_n(V) + beta_n(V))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

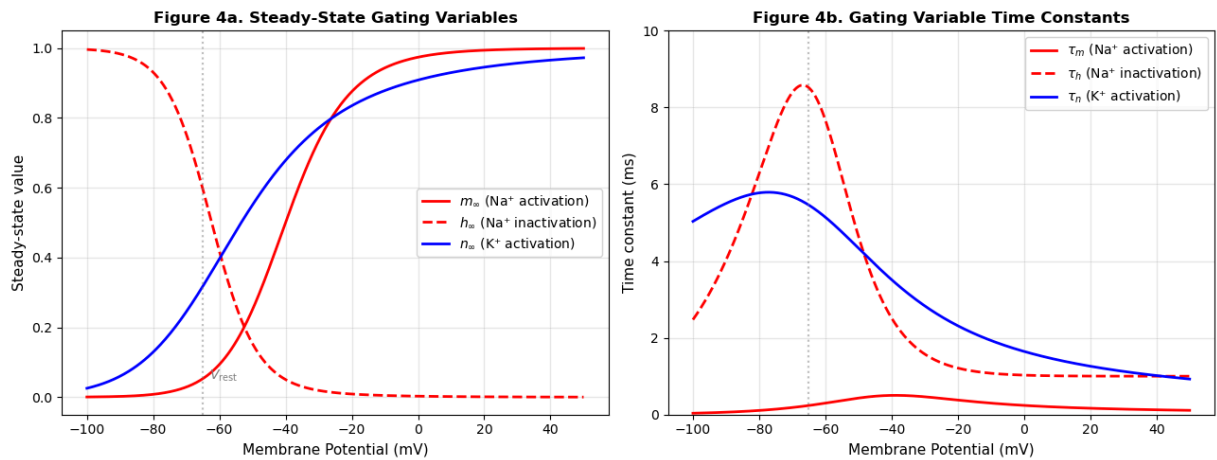
# Steady-state activation/inactivation curves
ax1 = axes[0]
ax1.plot(V, m_inf, 'r-', lw=2, label=r'$m_{\infty}$ (Na+ activation)')
ax1.plot(V, h_inf, 'r--', lw=2, label=r'$h_{\infty}$ (Na+ inactivation)')
ax1.plot(V, n_inf, 'b-', lw=2, label=r'$n_{\infty}$ (K+ activation)')
ax1.axvline(x=-65, color='gray', linestyle=':', alpha=0.5)
ax1.text(-63, 0.05, r'$V_{\mathrm{rest}}$', fontsize=10, color='gray')
ax1.set_xlabel('Membrane Potential (mV)', fontsize=11)
ax1.set_ylabel('Steady-state value', fontsize=11)
ax1.set_title('Figure 4a. Steady-State Gating Variables', fontweight='bold')
ax1.legend(fontsize=10, loc='center right')
ax1.grid(True, alpha=0.3)
ax1.set_ylim(-0.05, 1.05)

# Time constants
ax2 = axes[1]
ax2.plot(V, tau_m, 'r-', lw=2, label=r'$\tau_m$ (Na+ activation)')
ax2.plot(V, tau_h, 'r--', lw=2, label=r'$\tau_h$ (Na+ inactivation)')
ax2.plot(V, tau_n, 'b-', lw=2, label=r'$\tau_n$ (K+ activation)')
ax2.axvline(x=-65, color='gray', linestyle=':', alpha=0.5)
ax2.set_xlabel('Membrane Potential (mV)', fontsize=11)
ax2.set_ylabel('Time constant (ms)', fontsize=11)
ax2.set_title('Figure 4b. Gating Variable Time Constants', fontweight='bold')
ax2.legend(fontsize=10)
ax2.grid(True, alpha=0.3)
ax2.set_ylim(0, 10)

plt.tight_layout()
plt.show()

print("KEY OBSERVATIONS:")
print("-" * 60)
print("• m (Na+ activation) has the FASTEST kinetics ( $\tau < 1$  ms)")
print("  → Na+ channels open very quickly upon depolarization")
print("• h (Na+ inactivation) is SLOWER ( $\tau \sim 1-8$  ms)")
print("  → Na+ channels close with a delay after opening")
print("• n (K+ activation) is also SLOW ( $\tau \sim 1-6$  ms)")
print("  → K+ channels open slowly, enabling repolarization")
print("• This TIME SEPARATION is the key to the action potential!")

```



KEY OBSERVATIONS:

- m (Na^+ activation) has the FASTEST kinetics ($\tau < 1$ ms)
→ Na^+ channels open very quickly upon depolarization
- h (Na^+ inactivation) is SLOWER ($\tau \sim 1\text{--}8$ ms)
→ Na^+ channels close with a delay after opening
- n (K^+ activation) is also SLOW ($\tau \sim 1\text{--}6$ ms)
→ K^+ channels open slowly, enabling repolarization
- This TIME SEPARATION is the key to the action potential!

```
In [35]: import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import numpy as np

fig, axes = plt.subplots(1, 4, figsize=(14, 4))

titles = ['① Resting\n(V ≈ -65 mV)', '② Depolarization\n(V → +40 mV)',
          '③ Peak / Inactivation', '④ Repolarization\n(V → -65 mV)']

# Channel states: (m-gate open?, h-gate open?, n-gate open?)
# Na+ channel: needs m=open AND h=open
# K+ channel: needs n=open
states = [
    {'m': False, 'h': True, 'n': False, 'na_flow': False, 'k_flow': False},
    {'m': True, 'h': True, 'n': False, 'na_flow': True, 'k_flow': False},
    {'m': True, 'h': False, 'n': True, 'na_flow': False, 'k_flow': True}, #
    {'m': False, 'h': False, 'n': True, 'na_flow': False, 'k_flow': True},
]

for idx, (ax, title, state) in enumerate(zip(axes, titles, states)):
    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(title, fontsize=9, fontweight='bold', pad=8)

    # Membrane (horizontal band)
    membrane = FancyBboxPatch((0.5, 3.8), 9, 2.4, boxstyle="round,pad=0.1",
                               facecolor='#f5e6c8', edgecolor='#b8860b', lw=
    ax.add_patch(membrane)
```

```

# Na+ channel (left)
na_color = '#ff6666' if state['na_flow'] else '#ffcccc'
na_border = 'red' if state['na_flow'] else '#cc9999'
na_ch = FancyBboxPatch((1.5, 3.5), 2.5, 3.0, boxstyle="round,pad=0.1",
                      facecolor=na_color, edgecolor=na_border, lw=2)
ax.add_patch(na_ch)
ax.text(2.75, 5.6, 'Na+', fontsize=10, ha='center', fontweight='bold', c

# m gate
m_label = 'OPEN' if state['m'] else 'CLOSED'
m_color = '#00aa00' if state['m'] else '#cc0000'
ax.text(2.75, 4.8, f'm: {m_label}', fontsize=7, ha='center', color=m_col

# h gate
h_label = 'OPEN' if state['h'] else 'CLOSED'
h_color = '#00aa00' if state['h'] else '#cc0000'
ax.text(2.75, 4.2, f'h: {h_label}', fontsize=7, ha='center', color=h_col

# Na+ flow arrow
if state['na_flow']:
    ax.annotate('', xy=(2.75, 3.2), xytext=(2.75, 7.2),
               arrowprops=dict(arrowstyle='->', color='red', lw=2.5))
    ax.text(2.75, 7.5, 'Na+ IN', fontsize=8, ha='center', color='red', f

# K+ channel (right)
k_color = '#6699ff' if state['k_flow'] else '#cce0ff'
k_border = 'blue' if state['k_flow'] else '#9999cc'
k_ch = FancyBboxPatch((6.0, 3.5), 2.5, 3.0, boxstyle="round,pad=0.1",
                      facecolor=k_color, edgecolor=k_border, lw=2)
ax.add_patch(k_ch)
ax.text(7.25, 5.6, 'K+', fontsize=10, ha='center', fontweight='bold', cc

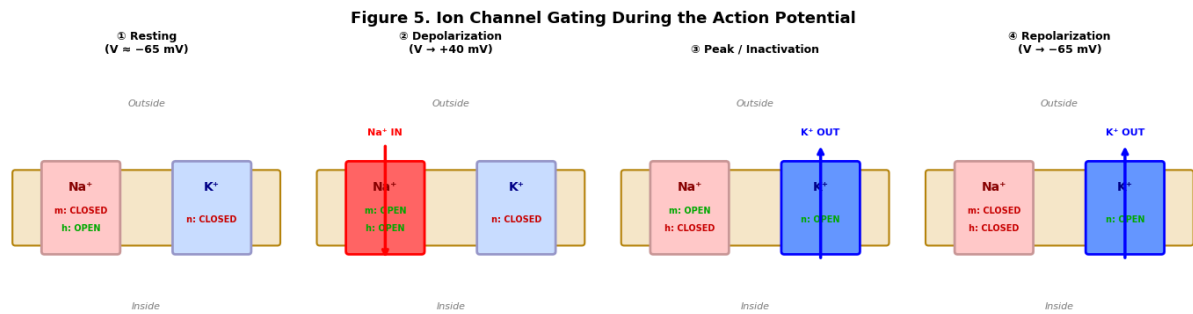
# n gate
n_label = 'OPEN' if state['n'] else 'CLOSED'
n_color = '#00aa00' if state['n'] else '#cc0000'
ax.text(7.25, 4.5, f'n: {n_label}', fontsize=7, ha='center', color=n_col

# K+ flow arrow
if state['k_flow']:
    ax.annotate('', xy=(7.25, 7.2), xytext=(7.25, 3.2),
               arrowprops=dict(arrowstyle='->', color='blue', lw=2.5))
    ax.text(7.25, 7.5, 'K+ OUT', fontsize=8, ha='center', color='blue',

# Labels
ax.text(5, 8.5, 'Outside', fontsize=8, ha='center', color='gray', style=
ax.text(5, 1.5, 'Inside', fontsize=8, ha='center', color='gray', style=

plt.suptitle('Figure 5. Ion Channel Gating During the Action Potential',
            fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Interactive Exploration: Channel Gating During an Action Potential

The static diagram above (Figure 5) shows four discrete snapshots. Use the **time slider** below to scrub continuously through an action potential and watch:

- The Na⁺ and K⁺ channel gates **open and close in real time**
- Ion flow arrows appear when conductance is significant
- The corresponding **membrane potential** and **gating variable** traces update with a time marker

Try dragging the time slider slowly from 0 to 30 ms to see the full sequence of events.

```
In [ ]: import ipywidgets as widgets
from IPython.display import display
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, Rectangle

# -----
# Self-contained HH solver (independent of the class defined later)
# -----

def _hh_solve(t_max=30, dt=0.01, I_ext_func=None):
    if I_ext_func is None:
        I_ext_func = lambda t: 0.0

    g_Na, g_K, g_L = 120.0, 36.0, 0.3
    E_Na, E_K, E_L = 50.0, -77.0, -54.4
    V0 = -65.0

    am = lambda V: 0.1 * (V + 40) / (1 - np.exp(-(V + 40) / 10))
    bm = lambda V: 4.0 * np.exp(-(V + 65) / 18)
    ah = lambda V: 0.07 * np.exp(-(V + 65) / 20)
    bh = lambda V: 1.0 / (1 + np.exp(-(V + 35) / 10))
    an = lambda V: 0.01 * (V + 55) / (1 - np.exp(-(V + 55) / 10))
    bn = lambda V: 0.125 * np.exp(-(V + 65) / 80)

    m0 = am(V0) / (am(V0) + bm(V0))
    h0 = ah(V0) / (ah(V0) + bh(V0))
    n0 = an(V0) / (an(V0) + bn(V0))
```

```

def deriv(y, t, I_fn):
    V, m, h, n = y
    return [
        (I_fn(t) - g_Na*m**3*h*(V - E_Na)
         - g_K*n**4*(V - E_K)
         - g_L*(V - E_L)),
        am(V)*(1-m) - bm(V)*m,
        ah(V)*(1-h) - bh(V)*h,
        an(V)*(1-n) - bn(V)*n,
    ]

t = np.arange(0, t_max, dt)
sol = odeint(deriv, [V0, m0, h0, n0], t, args=(I_ext_func,))
return t, sol[:, 0], sol[:, 1], sol[:, 2], sol[:, 3]

# -----
# Drawing helpers
# -----
def _phase_label(V, V_prev, V_next, dt):
    dVdt = (V_next - V_prev) / (2 * dt)
    if abs(V - (-65)) < 3 and abs(dVdt) < 10:
        return "Resting (V ~ -65 mV)"
    if dVdt > 10:
        return "Depolarization (V rising)"
    if V > -5:
        return "Peak / Inactivation"
    if dVdt < -10:
        return "Repolarization (V falling)"
    if V < -68:
        return "Hyperpolarization (undershoot)"
    return "Recovery"

def _gate_bar(ax, x, y, w, val, label, color):
    ax.add_patch(Rectangle((x, y), w, 0.45,
                           facecolor='#e8e8e8', edgecolor='#aaa', lw=0.5))
    ax.add_patch(Rectangle((x, y), w * val, 0.45,
                           facecolor=color, alpha=0.55))
    ax.text(x - 0.3, y + 0.22, label,
            fontsize=10, ha='right', va='center', fontweight='bold', color='c')
    ax.text(x + w + 0.2, y + 0.22, f'{val:.2f}',
            fontsize=9, ha='left', va='center')

# -----
# Main callback
# -----
_cg_cache = {}

def _channel_gating_cb(I_amp, stim_start, stim_dur, t_int):
    t_point = t_int / 10.0
    stim_end = stim_start + stim_dur
    key = (I_amp, stim_start, stim_dur)
    if key not in _cg_cache:
        _cg_cache.clear()
        _cg_cache[key] = _hh_solve(
            t_max=30,

```

```

        I_ext_func=lambda t: I_amp if stim_start <= t <= stim_end else 0
    )
    t, V, m, h, n = _cg_cache[key]

    idx = np.argmin(np.abs(t - t_point))
    idx_lo = max(idx - 1, 0)
    idx_hi = min(idx + 1, len(t) - 1)
    Vn, mn, hn, nn = V[idx], m[idx], h[idx], n[idx]
    g_na = mn**3 * hn
    g_k = nn**4
    phase = _phase_label(Vn, V[idx_lo], V[idx_hi], 0.01)

    fig = plt.figure(figsize=(13, 11))
    gs = fig.add_gridspec(3, 1, height_ratios=[2.8, 1, 1], hspace=0.38)

    # — Top: channel diagram —————
    ax = fig.add_subplot(gs[0])
    ax.set_xlim(0, 20)
    ax.set_ylim(0, 10)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(
        f'Ion Channel Gating — {phase}\n'
        f'V = {Vn:.1f} mV      t = {t_point:.1f} ms',
        fontsize=13, fontweight='bold', pad=10,
    )

    # Membrane band
    ax.add_patch(FancyBboxPatch(
        (0.5, 3.5), 19, 2.5, boxstyle="round,pad=0.1",
        facecolor='#f5e6c8', edgecolor='#b8860b', lw=1.5))
    ax.text(10, 8.8, 'Outside (extracellular)',
            fontsize=10, ha='center', color='gray', style='italic')
    ax.text(10, 1.2, 'Inside (intracellular)',
            fontsize=10, ha='center', color='gray', style='italic')

    # — Na+ channel —
    na_on = g_na > 0.02
    na_fill = (1.0, max(0.35, 1 - min(g_na, 1)*0.65),
               max(0.35, 1 - min(g_na, 1)*0.65))
    ax.add_patch(FancyBboxPatch(
        (2, 2.9), 4.2, 3.8, boxstyle="round,pad=0.15",
        facecolor=na_fill, edgecolor='red' if na_on else '#cc9999', lw=2))
    ax.text(4.1, 6.2, 'Na+ Channel',
            fontsize=11, ha='center', fontweight='bold', color='darkred')

    m_st = "OPEN" if mn > 0.5 else "CLOSED"
    h_st = "OPEN" if hn > 0.5 else "CLOSED"
    ax.text(4.1, 5.3, f'm = {mn:.2f} ({m_st})', fontsize=9, ha='center',
            fontweight='bold', color='#00aa00' if mn > 0.5 else '#cc0000')
    ax.text(4.1, 4.6, f'h = {hn:.2f} ({h_st})', fontsize=9, ha='center',
            fontweight='bold', color='#00aa00' if hn > 0.5 else '#cc0000')
    ax.text(4.1, 3.55, f'g_Na m³h = {120*g_na:.1f} mS/cm²',
            fontsize=8, ha='center', color='#555')

    if na_on:

```

```

        alph = min(1.0, g_na * 3)
        ax.annotate('', xy=(4.1, 2.4), xytext=(4.1, 7.4),
                    arrowprops=dict(arrowstyle='->', color=(1, 0, 0, alph),
ax.text(4.1, 7.7, 'Na+ IN', fontsize=9, ha='center',
        color='red', fontweight='bold', alpha=alph)

# — K+ channel —
k_on = g_k > 0.02
k_fill = (max(0.35, 1 - min(g_k, 1)*0.65),
          max(0.35, 1 - min(g_k, 1)*0.65), 1.0)
ax.add_patch(FancyBboxPatch(
    (13.8, 2.9), 4.2, 3.8, boxstyle="round,pad=0.15",
    facecolor=k_fill, edgecolor='blue' if k_on else '#9999cc', lw=2))
ax.text(15.9, 6.2, 'K+ Channel',
        fontsize=11, ha='center', fontweight='bold', color='darkblue')

n_st = "OPEN" if nn > 0.5 else "CLOSED"
ax.text(15.9, 5.0, f'n = {nn:.2f} ({n_st})', fontsize=9, ha='center',
        fontweight='bold', color='#00aa00' if nn > 0.5 else '#cc0000')
ax.text(15.9, 3.55, f'g_K n^4 = {36*g_k:.1f} mS/cm^2',
        fontsize=8, ha='center', color='#555')

if k_on:
    alph = min(1.0, g_k * 3)
    ax.annotate('', xy=(15.9, 7.4), xytext=(15.9, 2.4),
                arrowprops=dict(arrowstyle='->', color=(0, 0, 1, alph),
ax.text(15.9, 7.7, 'K+ OUT', fontsize=9, ha='center',
        color='blue', fontweight='bold', alpha=alph)

# — Gate-value bars (centre) —
bx, bw = 8.5, 3
_gate_bar(ax, bx, 5.6, bw, mn, 'm', 'red')
_gate_bar(ax, bx, 4.8, bw, hn, 'h', '#cc4444')
_gate_bar(ax, bx, 4.0, bw, nn, 'n', 'blue')

# — Middle: membrane potential —
ax_v = fig.add_subplot(gs[1])
ax_v.plot(t, V, 'k-', lw=2)
ax_v.axvline(t_point, color='red', lw=1.5, ls='--', alpha=.7)
ax_v.plot(t_point, Vn, 'ro', ms=8, zorder=5)
ax_v.axhline(-65, color='gray', ls=':', alpha=.4)
ax_v.axhline(-55, color='orange', ls='--', alpha=.4, label='Threshold')
ax_v.axvspan(stim_start, stim_end, alpha=.15, color='yellow', label='Sti
ax_v.set_ylabel('V (mV)', fontsize=11)
ax_v.set_title('Membrane Potential', fontsize=11, fontweight='bold')
ax_v.legend(loc='upper right', fontsize=8)
ax_v.grid(True, alpha=0.3)

# — Bottom: gating variables —
ax_g = fig.add_subplot(gs[2], sharex=ax_v)
ax_g.plot(t, m, 'r-', lw=2, label=r'$m$ (Na+ act.)')
ax_g.plot(t, h, 'r--', lw=2, label=r'$h$ (Na+ inact.)')
ax_g.plot(t, n, 'b-', lw=2, label=r'$n$ (K+ act.)')
ax_g.axvline(t_point, color='red', lw=1.5, ls='--', alpha=.7)
ax_g.plot(t_point, mn, 'ro', ms=6, zorder=5)
ax_g.plot(t_point, hn, 'rs', ms=6, zorder=5)

```

```

ax_g.plot(t_point, nn, 'bd', ms=6, zorder=5)
ax_g.set_xlabel('Time (ms)', fontsize=11)
ax_g.set_ylabel('Gate value', fontsize=11)
ax_g.set_ylim(-0.05, 1.05)
ax_g.set_title('Gating Variables', fontsize=11, fontweight='bold')
ax_g.legend(loc='right', fontsize=8)
ax_g.grid(True, alpha=0.3)

plt.show()

# -----
# Transport & widget definitions
# -----
style = {'description_width': '140px'}
sl_layout = widgets.Layout(width='380px')
kw_cg = dict(style=style, layout=sl_layout, continuous_update=False)

# --- Time control (integer 0-299 → 0.0-29.9 ms) ---
play_w = widgets.Play(
    value=0, min=0, max=299, step=1, interval=150,
    show_repeat=True, description='')

w_t_int = widgets.IntSlider(
    value=0, min=0, max=299, step=1, readout=False,
    continuous_update=True,
    layout=widgets.Layout(width='500px', flex='1'))

widgets.jslink((play_w, 'value'), (w_t_int, 'value'))

time_label = widgets.HTML('<b style="font-size:13px">t = 0.0 ms</b>')
def _lbl(change):
    time_label.value = f'<b style="font-size:13px">t = {change["new"]}/10:.1f
w_t_int.observe(_lbl, names='value')

# Speed selector (controls Play interval)
speed_dd = widgets.Dropdown(
    options=[('0.25x (very slow)', 400), ('0.5x', 200),
             ('1x', 150), ('2x', 75), ('4x', 40)],
    value=150, description='Speed:',
    style={'description_width': '50px'},
    layout=widgets.Layout(width='170px'))

def _spd(change):
    play_w.interval = change['new']
speed_dd.observe(_spd, names='value')

# --- Step / jump buttons ---
_bkw = dict(layout=widgets.Layout(width='42px', height='28px'))

btn_start = widgets.Button(description='|<<', tooltip='Jump to start',
btn_back_big = widgets.Button(description='<<', tooltip='Back 1.0 ms',
btn_back = widgets.Button(description='<', tooltip='Back 0.1 ms',
                           layout=widgets.Layout(width='34px', height='28
btn_fwd = widgets.Button(description='>', tooltip='Forward 0.1 ms',
                           layout=widgets.Layout(width='34px', height='28
btn_fwd_big = widgets.Button(description='>>', tooltip='Forward 1.0 ms',

```

```

btn_end      = widgets.Button(description='>>|', tooltip='Jump to end',

btn_start.on_click(lambda b: setattr(w_t_int, 'value', 0))
btn_back_big.on_click(lambda b: setattr(w_t_int, 'value', max(0, w_t_int.v
btn_back.on_click(lambda b: setattr(w_t_int, 'value', max(0, w_t_int.v
btn_fwd.on_click(lambda b: setattr(w_t_int, 'value', min(299, w_t_int.v
btn_fwd_big.on_click(lambda b: setattr(w_t_int, 'value', min(299, w_t_int.v
btn_end.on_click(lambda b: setattr(w_t_int, 'value', 299))

# --- Stimulus sliders ---
w_Ia = widgets.FloatSlider(value=10.0, min=-10, max=50, step=0.5,
                           description='I_ext (uA/cm2):', **kw_cg)
w_ss = widgets.FloatSlider(value=5.0, min=0, max=20, step=0.5,
                           description='Stim. onset (ms):', **kw_cg)
w_sd = widgets.FloatSlider(value=1.0, min=0.1, max=25, step=0.1,
                           description='Stim. duration (ms):', **kw_cg)

# --- Reset ---
def _reset_cg(b):
    w_t_int.value = 0
    w_Ia.value, w_ss.value, w_sd.value = 10.0, 5.0, 1.0
    speed_dd.value = 150
    _cg_cache.clear()

reset_cg = widgets.Button(description='Reset to defaults',
                           button_style='warning',
                           layout=widgets.Layout(width='160px'))
reset_cg.on_click(_reset_cg)

# --- Wire up interactive output ---
out_cg = widgets.interactive_output(_channel_gating_cb, dict(
    I_amp=w_Ia, stim_start=w_ss, stim_dur=w_sd, t_int=w_t_int))

# --- Layout ---
transport_row = widgets.HBox(
    [btn_start, btn_back_big, btn_back, play_w,
     btn_fwd, btn_fwd_big, btn_end, speed_dd, time_label],
    layout=widgets.Layout(align_items='center', gap='3px'))

timeline_row = widgets.HBox(
    [widgets.HTML('<span style="white-space:nowrap;font-size:12px">'
                  '0 ms&nbsp;</span>'),
     w_t_int,
     widgets.HTML('<span style="white-space:nowrap;font-size:12px">'
                  '&nbsp;&nbsp;&nbsp;29.9 ms</span>')],
    layout=widgets.Layout(align_items='center'))

stim_row = widgets.HBox([w_Ia, w_ss, w_sd],
                        layout=widgets.Layout(gap='12px'))

display(widgets.VBox([
    widgets.HTML(
        '<h3 style="margin-bottom:2px">Interactive Channel Gating Viewer</h3>'
        '<p style="color:#555;margin-top:0">'
        'Press <b>Play</b> or drag the timeline to scrub through the '
        'action potential. Use <b>&lt;</b> / <b>&gt;></b> for fine steps '

```

```

        'and <b>&lt;&lt;</b> / <b>&gt;&gt;</b> for 1 ms jumps.</p>'),
        transport_row,
        timeline_row,
        widgets.HTML('<hr style="margin:4px 0">'),
        stim_row,
        widgets.HBox([reset_cg]),
        out_cg,
    ]))

```

VBox(children=(HTML(value='<h3 style="margin-bottom:2px">Interactive Channel Gating Viewer</h3><p style="color...

```

In [ ]: # — Static snapshot for PDF / print export —————
# Generates representative frames of the channel gating viewer.

_t_snap, _V_snap, _m_snap, _h_snap, _n_snap = _hh_solve(
    t_max=30,
    I_ext_func=lambda t: 10.0 if 5 <= t <= 6 else 0.0,
)

_snap_times = [2.0, 7.0, 8.0, 9.5]
_snap_labels = [
    'Resting (V ~ -65 mV)',
    'Depolarization (V rising)',
    'Peak / Inactivation',
    'Repolarization (V falling)',
]

fig_snap = plt.figure(figsize=(14, 16))
outer = fig_snap.add_gridspec(3, 1, height_ratios=[3.2, 1, 1], hspace=0.32)

# — Top: four channel-state panels —
inner = outer[0].subgridspec(1, 4, wspace=0.35)

for col, (tp, label) in enumerate(zip(_snap_times, _snap_labels)):
    ax = fig_snap.add_subplot(inner[col])
    idx = np.argmin(np.abs(_t_snap - tp))
    Vn, mn, hn, nn = _V_snap[idx], _m_snap[idx], _h_snap[idx], _n_snap[idx]
    g_na, g_k = mn**3 * hn, nn**4

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(f'{label}\nV = {Vn:.0f} mV, t = {tp} ms',
                 fontsize=8, fontweight='bold', pad=6)

    # Membrane
    ax.add_patch(FancyBboxPatch((0.3, 3.5), 9.4, 2.5,
                                boxstyle="round,pad=0.1",
                                facecolor='#f5e6c8', edgecolor='#b8860b', lw=1))

    # Na+ channel
    na_on = g_na > 0.02
    na_c = (1, max(.35, 1-min(g_na,1)*.65), max(.35, 1-min(g_na,1)*.65))
    ax.add_patch(FancyBboxPatch((0.8, 3.2), 3.2, 3.2,

```

```

        boxstyle="round,pad=0.1",
        facecolor=na_c, edgecolor='red' if na_on else '#cc9999', lw=
ax.text(2.4, 5.8, 'Na+', fontsize=9, ha='center', fontweight='bold', col=
m_st = 'OPEN' if mn > 0.5 else 'CLOSED'
h_st = 'OPEN' if hn > 0.5 else 'CLOSED'
ax.text(2.4, 5.0, f'm: {m_st}', fontsize=6.5, ha='center',
        fontweight='bold', color='#00aa00' if mn > 0.5 else '#cc0000')
ax.text(2.4, 4.4, f'h: {h_st}', fontsize=6.5, ha='center',
        fontweight='bold', color='#00aa00' if hn > 0.5 else '#cc0000')
if na_on:
    alph = min(1., g_na*3)
    ax.annotate('', xy=(2.4, 2.8), xytext=(2.4, 7.0),
                arrowprops=dict(arrowstyle='->', color=(1,0,0,alpha), lw=
    ax.text(2.4, 7.3, 'Na+ IN', fontsize=7, ha='center',
            color='red', fontweight='bold', alpha=alph)

# K+ channel
k_on = g_k > 0.02
k_c = (max(.35, 1-min(g_k,1)*.65), max(.35, 1-min(g_k,1)*.65), 1)
ax.add_patch(FancyBboxPatch((6.0, 3.2), 3.2, 3.2,
                            boxstyle="round,pad=0.1",
                            facecolor=k_c, edgecolor='blue' if k_on else '#9999cc', lw=
ax.text(7.6, 5.8, 'K+', fontsize=9, ha='center', fontweight='bold', col=
n_st = 'OPEN' if nn > 0.5 else 'CLOSED'
ax.text(7.6, 4.7, f'n: {n_st}', fontsize=6.5, ha='center',
        fontweight='bold', color='#00aa00' if nn > 0.5 else '#cc0000')
if k_on:
    alph = min(1., g_k*3)
    ax.annotate('', xy=(7.6, 7.0), xytext=(7.6, 2.8),
                arrowprops=dict(arrowstyle='->', color=(0,0,1,alpha), lw=
    ax.text(7.6, 7.3, 'K+ OUT', fontsize=7, ha='center',
            color='blue', fontweight='bold', alpha=alph)

ax.text(5, 8.5, 'Outside', fontsize=7, ha='center', color='gray', style='
ax.text(5, 1.5, 'Inside', fontsize=7, ha='center', color='gray', style='

# — Middle: V(t) with markers —
ax_v = fig_snap.add_subplot(outer[1])
ax_v.plot(_t_snap, _V_snap, 'k-', lw=2)
for tp in _snap_times:
    idx = np.argmin(np.abs(_t_snap - tp))
    ax_v.plot(tp, _V_snap[idx], 'ro', ms=7, zorder=5)
    ax_v.axvline(tp, color='red', lw=0.8, ls='--', alpha=0.4)
ax_v.axhline(-65, color='gray', ls=':', alpha=.4)
ax_v.axhline(-55, color='orange', ls='--', alpha=.4, label='Threshold')
ax_v.axvspan(5, 6, alpha=.15, color='yellow', label='Stimulus')
ax_v.set_ylabel('V (mV)', fontsize=11)
ax_v.set_title('Membrane Potential (red dots = snapshots above)', fontsize=
ax_v.legend(loc='upper right', fontsize=8)
ax_v.grid(True, alpha=0.3)

# — Bottom: gating variables with markers —
ax_g = fig_snap.add_subplot(outer[2], sharex=ax_v)
ax_g.plot(_t_snap, _m_snap, 'r-', lw=2, label=r'$m$ (Na+ act.)')
ax_g.plot(_t_snap, _h_snap, 'r--', lw=2, label=r'$h$ (Na+ inact.)')
ax_g.plot(_t_snap, _n_snap, 'b-', lw=2, label=r'$n$ (K+ act.)')

```



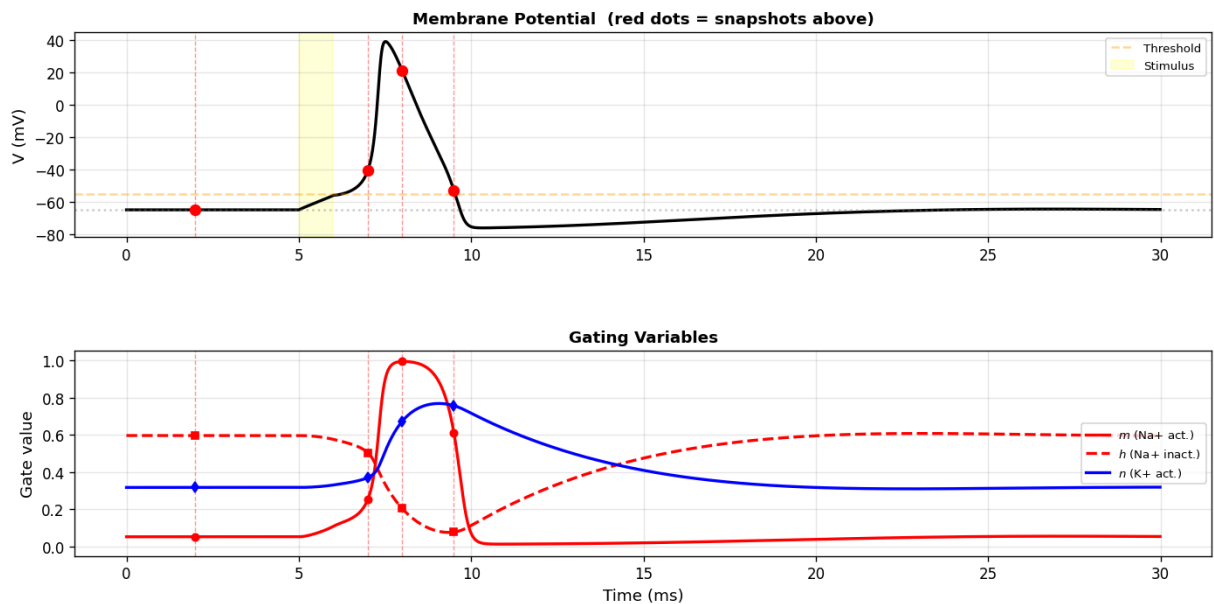
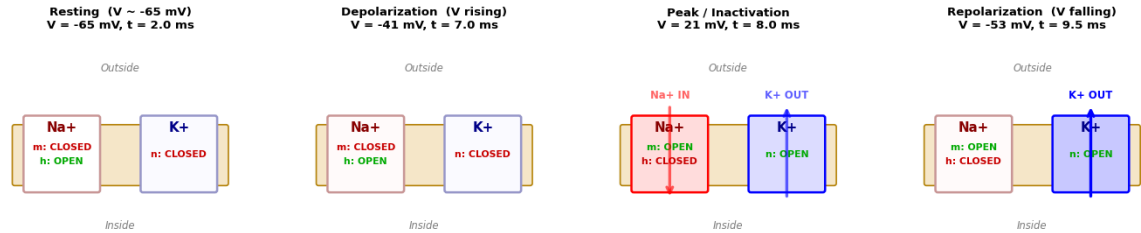
```

for tp in _snap_times:
    idx = np.argmin(np.abs(_t_snap - tp))
    ax_g.plot(tp, _m_snap[idx], 'ro', ms=5, zorder=5)
    ax_g.plot(tp, _h_snap[idx], 'rs', ms=5, zorder=5)
    ax_g.plot(tp, _n_snap[idx], 'bd', ms=5, zorder=5)
    ax_g.axvline(tp, color='red', lw=0.8, ls='--', alpha=0.4)
ax_g.set_xlabel('Time (ms)', fontsize=11)
ax_g.set_ylabel('Gate value', fontsize=11)
ax_g.set_ylim(-0.05, 1.05)
ax_g.set_title('Gating Variables', fontsize=11, fontweight='bold')
ax_g.legend(loc='right', fontsize=8)
ax_g.grid(True, alpha=0.3)

fig_snap.suptitle('Channel Gating Viewer – Static Snapshot (four key phases)',
                  fontsize=13, fontweight='bold', y=0.99)
plt.tight_layout()
plt.show()

```

Channel Gating Viewer — Static Snapshot (four key phases)



7. Python Implementation of the Hodgkin-Huxley Model

Below we implement the HH model as a clean Python class and simulate the response to an injected current pulse. We then visualize:

1. The **membrane potential** $V(t)$ — the action potential itself

2. The **gating variables** $m(t)$, $h(t)$, $n(t)$ — the opening/closing of ion channels
3. The **ionic currents** $I_{Na}(t)$, $I_K(t)$, $I_L(t)$ — the flow of ions
4. The **ionic conductances** $g_{Na} m^3 h$ and $g_K n^4$ — how channel permeability changes

```
In [37]: import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

class HodgkinHuxley:
    """Hodgkin-Huxley neuron model.

    Simulates the membrane potential dynamics and ion channel gating
    of a neuron based on the 1952 Hodgkin-Huxley equations.
    """

    def __init__(self):
        # Membrane capacitance ( $\mu F/cm^2$ )
        self.C_m = 1.0
        # Maximum conductances ( $mS/cm^2$ )
        self.g_Na = 120.0 # Sodium
        self.g_K = 36.0 # Potassium
        self.g_L = 0.3 # Leak
        # Reversal potentials (mV)
        self.E_Na = 50.0
        self.E_K = -77.0
        self.E_L = -54.4

    # --- Voltage-dependent rate constants ---
    @staticmethod
    def alpha_m(V): return 0.1 * (V + 40) / (1 - np.exp(-(V + 40) / 10))
    @staticmethod
    def beta_m(V): return 4.0 * np.exp(-(V + 65) / 18)
    @staticmethod
    def alpha_h(V): return 0.07 * np.exp(-(V + 65) / 20)
    @staticmethod
    def beta_h(V): return 1.0 / (1 + np.exp(-(V + 35) / 10))
    @staticmethod
    def alpha_n(V): return 0.01 * (V + 55) / (1 - np.exp(-(V + 55) / 10))
    @staticmethod
    def beta_n(V): return 0.125 * np.exp(-(V + 65) / 80)

    def derivatives(self, state, t, I_ext_func):
        """Compute dV/dt, dm/dt, dh/dt, dn/dt."""
        V, m, h, n = state
        I_ext = I_ext_func(t)

        # Ionic currents
        I_Na = self.g_Na * m**3 * h * (V - self.E_Na)
        I_K = self.g_K * n**4 * (V - self.E_K)
        I_L = self.g_L * (V - self.E_L)

        # Differential equations
        dVdt = (I_ext - I_Na - I_K - I_L) / self.C_m
        dmdt = self.alpha_m(V) * (1 - m) - self.beta_m(V) * m
        dhdt = self.alpha_h(V) * (1 - h) - self.beta_h(V) * h
```

```

        dndt = self.alpha_n(V) * (1 - n) - self.beta_n(V) * n

    return [dVdt, dmdt, dhdt, dndt]

def steady_state(self, V):
    """Return steady-state gating variables at voltage V."""
    m0 = self.alpha_m(V) / (self.alpha_m(V) + self.beta_m(V))
    h0 = self.alpha_h(V) / (self.alpha_h(V) + self.beta_h(V))
    n0 = self.alpha_n(V) / (self.alpha_n(V) + self.beta_n(V))
    return m0, h0, n0

def simulate(self, t_max=50, dt=0.01, I_ext_func=None, V_rest=-65.0):
    """Run simulation and return time, states, and derived quantities."""
    if I_ext_func is None:
        I_ext_func = lambda t: 0.0

    # Initial conditions: resting state
    m0, h0, n0 = self.steady_state(V_rest)
    state0 = [V_rest, m0, h0, n0]

    # Time array
    t = np.arange(0, t_max, dt)

    # Solve ODE system
    sol = odeint(self.derivatives, state0, t, args=(I_ext_func,))

    # Unpack
    V = sol[:, 0]
    m = sol[:, 1]
    h = sol[:, 2]
    n = sol[:, 3]

    # Compute ionic currents
    I_Na = self.g_Na * m**3 * h * (V - self.E_Na)
    I_K = self.g_K * n**4 * (V - self.E_K)
    I_L = self.g_L * (V - self.E_L)

    # Compute conductances
    g_Na_t = self.g_Na * m**3 * h
    g_K_t = self.g_K * n**4

    return {
        't': t, 'V': V, 'm': m, 'h': h, 'n': n,
        'I_Na': I_Na, 'I_K': I_K, 'I_L': I_L,
        'g_Na_t': g_Na_t, 'g_K_t': g_K_t
    }

# — Simulation: Single current pulse —————
hh = HodgkinHuxley()

# External current: 10 µA/cm² pulse from t=5 to t=6 ms
def I_pulse(t):
    return 10.0 if 5 <= t <= 6 else 0.0

results = hh.simulate(t_max=30, I_ext_func=I_pulse)
t = results['t']

```

```

# — Four-panel visualization —————
fig, axes = plt.subplots(4, 1, figsize=(11, 14), sharex=True)

# Panel 1: Membrane potential
ax = axes[0]
ax.plot(t, results['V'], 'k-', lw=2)
ax.axhline(y=-65, color='gray', linestyle=':', alpha=0.5, label='Resting pot')
ax.axhline(y=-55, color='orange', linestyle='--', alpha=0.5, label='Threshold')
ax.axvspan(5, 6, alpha=0.15, color='yellow', label='Stimulus')
ax.set_ylabel('Membrane\nPotential (mV)', fontsize=11)
ax.set_title('Figure 6. Hodgkin-Huxley Simulation: Single Action Potential',
ax.legend(loc='upper right', fontsize=9)
ax.grid(True, alpha=0.3)

# Panel 2: Gating variables
ax = axes[1]
ax.plot(t, results['m'], 'r-', lw=2, label=r'$m$ (Na+ activation)')
ax.plot(t, results['h'], 'r--', lw=2, label=r'$h$ (Na+ inactivation)')
ax.plot(t, results['n'], 'b-', lw=2, label=r'$n$ (K+ activation)')
ax.set_ylabel('Gating\nVariable', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)
ax.set_ylim(-0.05, 1.05)

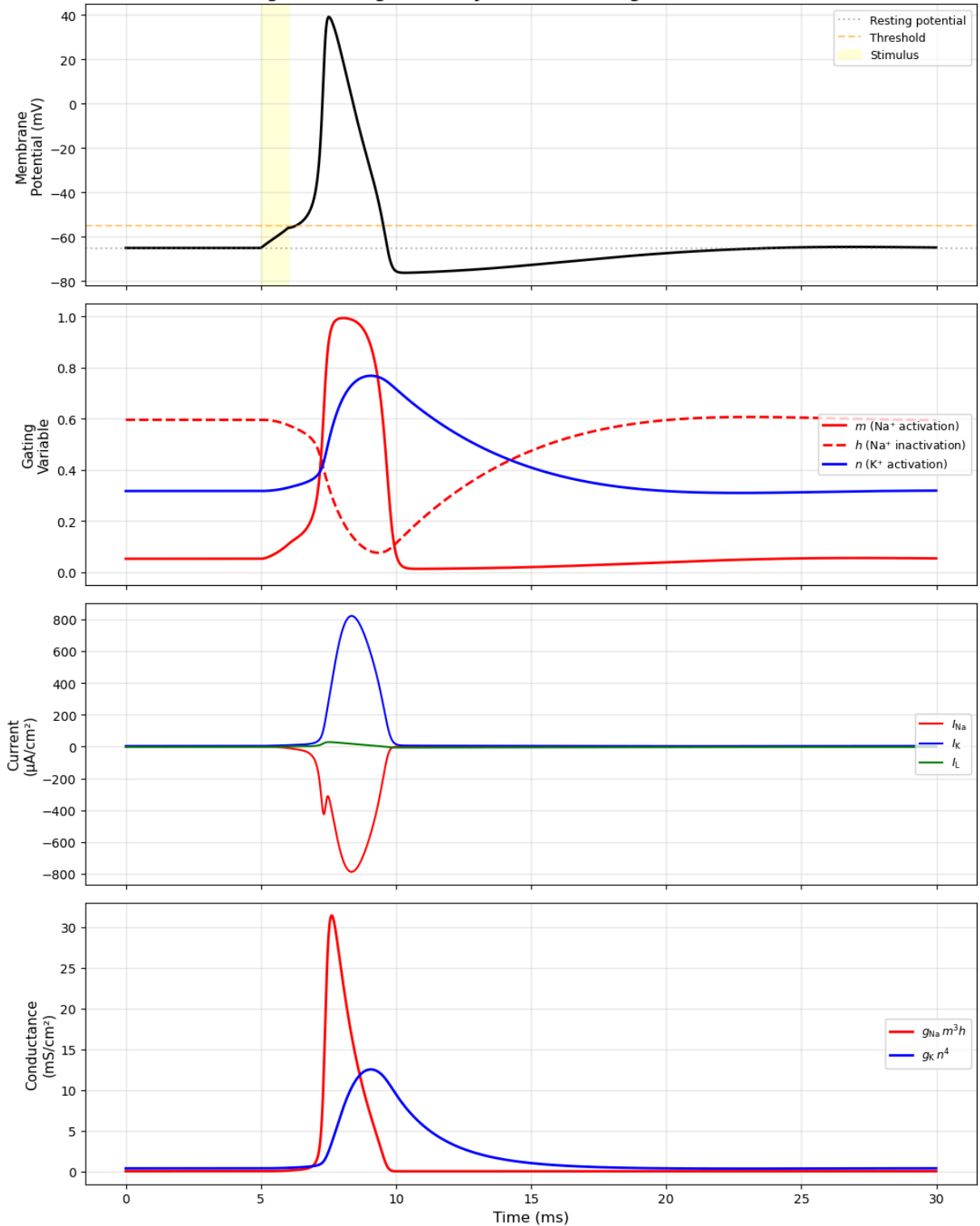
# Panel 3: Ionic currents
ax = axes[2]
ax.plot(t, results['I_Na'], 'r-', lw=1.5, label=r'$I_{\mathrm{Na}}$')
ax.plot(t, results['I_K'], 'b-', lw=1.5, label=r'$I_{\mathrm{K}}$')
ax.plot(t, results['I_L'], 'g-', lw=1.5, label=r'$I_{\mathrm{L}}$')
ax.set_ylabel('Current\n(μA/cm2)', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

# Panel 4: Conductances
ax = axes[3]
ax.plot(t, results['g_Na_t'], 'r-', lw=2, label=r'$g_{\mathrm{Na}}\backslash, m^3 h$')
ax.plot(t, results['g_K_t'], 'b-', lw=2, label=r'$g_{\mathrm{K}}\backslash, n^4$')
ax.set_ylabel('Conductance\n(mS/cm2)', fontsize=11)
ax.set_xlabel('Time (ms)', fontsize=12)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Figure 6. Hodgkin-Huxley Simulation: Single Action Potential



Interpreting the simulation results (Figure 6)

The four panels above tell the complete story of how a single action potential is generated:

Panel 1 — Membrane Potential: The neuron rests at -65 mV. A brief current pulse (yellow shading, 5–6 ms) depolarizes the membrane past threshold (~-55 mV),

triggering a rapid spike to $\sim +40$ mV (depolarization) followed by a return to rest (repolarization) and a brief undershoot (hyperpolarization). This all-or-nothing spike is the **action potential**.

Panel 2 — Gating Variables: The action potential is orchestrated by the precise timing of three gating processes:

- m (Na^+ activation, red solid) **opens rapidly** — this is the fastest gate, responsible for the explosive depolarization
- h (Na^+ inactivation, red dashed) **closes with a delay** — this shuts down Na^+ current even while the membrane is depolarized
- n (K^+ activation, blue) **opens slowly** — delayed K^+ efflux drives repolarization

The key insight is the **temporal separation**: m is fast, while h and n are slow. This creates a brief window where Na^+ channels are open (m high, h still high) before they inactivate (h drops) and K^+ channels take over (n rises).

Panel 3 — Ionic Currents: The large inward (negative) Na^+ current drives depolarization, while the delayed outward K^+ current drives repolarization. The leak current remains small throughout.

Panel 4 — Conductances: Shows how the *effective* Na^+ conductance ($g_{\text{Na}} m^3 h$) rises and falls rapidly (transient), while the K^+ conductance ($g_{\text{K}} n^4$) rises more slowly and persists longer (sustained). This transient-vs-sustained pattern is the hallmark of the HH model.

Interactive Exploration: Hodgkin-Huxley Parameters

Use the sliders below to explore how each biophysical parameter shapes the action potential. Try these experiments:

- **Block Na^+ channels** (simulating tetrodotoxin / TTX): reduce g_{Na} toward zero — what happens to the spike?
- **Block K^+ channels** (simulating tetraethylammonium / TEA): reduce g_{K} toward zero — how does repolarization change?
- **Increase membrane capacitance**: raise C_m — the membrane charges more slowly. Does the spike broaden?
- **Sustained stimulus**: set a long stimulus duration with moderate I_{ext} — can you trigger repetitive firing?
- **Subthreshold stimulus**: try $I_{\text{ext}} \approx 3 \mu\text{A}/\text{cm}^2$ — does an action potential fire?

```
In [38]: import ipywidgets as widgets
from IPython.display import display
import numpy as np
import matplotlib.pyplot as plt
```

```

# -----
# Plotting callback
# -----
def hh_interactive_plot(g_Na, g_K, g_L, E_Na, E_K, E_L, C_m,
                        I_amp, stim_start, stim_dur, t_max, preset):

    # Apply presets (override slider values when a preset is selected)
    if preset == "TTX (Na+ block)":
        g_Na = 0.0
    elif preset == "TEA (K+ block)":
        g_K = 0.0
    elif preset == "TTX + TEA":
        g_Na, g_K = 0.0, 0.0

    hh = HodgkinHuxley()
    hh.g_Na, hh.g_K, hh.g_L = g_Na, g_K, g_L
    hh.E_Na, hh.E_K, hh.E_L = E_Na, E_K, E_L
    hh.C_m = C_m

    stim_end = stim_start + stim_dur
    results = hh.simulate(
        t_max=t_max,
        I_ext_func=lambda t: I_amp if stim_start <= t <= stim_end else 0.0,
    )
    t = results['t']

    fig, axes = plt.subplots(4, 1, figsize=(11, 13), sharex=True)
    fig.suptitle('Interactive Hodgkin-Huxley Simulation',
                 fontsize=13, fontweight='bold', y=0.98)

    # Panel 1 – Membrane potential
    ax = axes[0]
    ax.plot(t, results['V'], 'k-', lw=2)
    ax.axhline(y=-65, color='gray', ls=':', alpha=.5, label='Resting potential')
    ax.axhline(y=-55, color='orange', ls='--', alpha=.5, label='Threshold')
    ax.axvspan(stim_start, stim_end, alpha=.15, color='yellow', label='Stimulus')
    ax.set_ylabel('Membrane\nPotential (mV)', fontsize=11)
    ax.legend(loc='upper right', fontsize=9)
    ax.grid(True, alpha=0.3)

    # Panel 2 – Gating variables
    ax = axes[1]
    ax.plot(t, results['m'], 'r-', lw=2, label=r'$m$ (Na+ activation)')
    ax.plot(t, results['h'], 'r--', lw=2, label=r'$h$ (Na+ inactivation)')
    ax.plot(t, results['n'], 'b-', lw=2, label=r'$n$ (K+ activation)')
    ax.set_ylabel('Gating\nVariable', fontsize=11)
    ax.legend(loc='right', fontsize=9)
    ax.set_ylim(-0.05, 1.05)
    ax.grid(True, alpha=0.3)

    # Panel 3 – Ionic currents
    ax = axes[2]
    ax.plot(t, results['I_Na'], 'r-', lw=1.5, label=r'$I_{\mathrm{Na}}$')
    ax.plot(t, results['I_K'], 'b-', lw=1.5, label=r'$I_{\mathrm{K}}$')
    ax.plot(t, results['I_L'], 'g-', lw=1.5, label=r'$I_{\mathrm{L}}$')
    ax.set_ylabel('Current\n(uA/cm2)', fontsize=11)

```



```

ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

# Panel 4 – Conductances
ax = axes[3]
ax.plot(t, results['g_Na_t'], 'r-', lw=2, label=r'$g_{\mathrm{Na}}\backslash,m^3\ h$')
ax.plot(t, results['g_K_t'], 'b-', lw=2, label=r'$g_{\mathrm{K}}\backslash,n^4$')
ax.set_ylabel('Conductance\n(mS/cm2)', fontsize=11)
ax.set_xlabel('Time (ms)', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# -----
# Widget definitions
# -----

style = {'description_width': '160px'}
layout = widgets.Layout(width='430px')
kw = dict(style=style, layout=layout, continuous_update=False)

# Conductances
w_g_Na = widgets.FloatSlider(value=120.0, min=0, max=300, step=1.0,
                             description='g_Na (mS/cm2):', **kw)
w_g_K = widgets.FloatSlider(value=36.0, min=0, max=100, step=0.5,
                             description='g_K (mS/cm2):', **kw)
w_g_L = widgets.FloatSlider(value=0.3, min=0, max=2.0, step=0.05,
                             description='g_L (mS/cm2):', **kw)

# Reversal potentials
w_E_Na = widgets.FloatSlider(value=50.0, min=20, max=80, step=1.0,
                              description='E_Na (mV):', **kw)
w_E_K = widgets.FloatSlider(value=-77.0, min=-100, max=-50, step=1.0,
                              description='E_K (mV):', **kw)
w_E_L = widgets.FloatSlider(value=-54.4, min=-70, max=-40, step=0.5,
                              description='E_L (mV):', **kw)

# Membrane capacitance
w_C_m = widgets.FloatSlider(value=1.0, min=0.5, max=5.0, step=0.1,
                             description='C_m (uF/cm2):', **kw)

# Stimulus
w_I = widgets.FloatSlider(value=10.0, min=-20, max=50, step=0.5,
                           description='I_ext (uA/cm2):', **kw)
w_on = widgets.FloatSlider(value=5.0, min=0, max=40, step=0.5,
                            description='Stim. onset (ms):', **kw)
w_dur = widgets.FloatSlider(value=1.0, min=0.1, max=40, step=0.1,
                             description='Stim. duration (ms):', **kw)
w_tmax = widgets.FloatSlider(value=30.0, min=10, max=100, step=5.0,
                              description='Simulation time (ms):', **kw)

# Presets dropdown
w_preset = widgets.Dropdown(
    options=['Custom', 'TTX (Na+ block)', 'TEA (K+ block)', 'TTX + TEA'],
    value='Custom',

```

```

        description='Preset:',
        style=style, layout=layout,
    )

# Reset button
def _reset(b):
    w_g_Na.value, w_g_K.value, w_g_L.value = 120.0, 36.0, 0.3
    w_E_Na.value, w_E_K.value, w_E_L.value = 50.0, -77.0, -54.4
    w_C_m.value = 1.0
    w_I.value, w_on.value, w_dur.value, w_tmax.value = 10.0, 5.0, 1.0, 30.0
    w_preset.value = 'Custom'

reset_btn = widgets.Button(description='Reset to defaults',
                           button_style='warning',
                           layout=widgets.Layout(width='200px'))
reset_btn.on_click(_reset)

# -----
# Layout & display
# -----
out = widgets.interactive_output(hh_interactive_plot, dict(
    g_Na=w_g_Na, g_K=w_g_K, g_L=w_g_L,
    E_Na=w_E_Na, E_K=w_E_K, E_L=w_E_L,
    C_m=w_C_m,
    I_amp=w_I, stim_start=w_on, stim_dur=w_dur,
    t_max=w_tmax, preset=w_preset,
))

controls = widgets.VBox([
    widgets.HTML(
        '<h3 style="margin-bottom:4px">Interactive Hodgkin-Huxley Simulator<
        <p style="color:#555; margin-top:0">Drag any slider and release to
    ),
    widgets.HBox([
        widgets.VBox([
            widgets.HTML('<b>Conductances</b>'),
            w_g_Na, w_g_K, w_g_L,
            widgets.HTML('<b>Membrane</b>'),
            w_C_m,
        ]),
        widgets.VBox([
            widgets.HTML('<b>Reversal Potentials</b>'),
            w_E_Na, w_E_K, w_E_L,
            widgets.HTML('<b>Stimulus</b>'),
            w_I, w_on, w_dur, w_tmax,
        ]),
    ]),
    widgets.HBox([w_preset, reset_btn]),
])

display(widgets.VBox([controls, out]))

```

VBox(children=(VBox(children=(HTML(value='<h3 style="margin-bottom:4px">Interactive Hodgkin-Huxley Simulator</...)

```

In [ ]: # — Static snapshot for PDF / print export —————
# Reproduces the Interactive HH Simulator output with default parameters.

hh_snap = HodgkinHuxley()
res_snap = hh_snap.simulate(
    t_max=30,
    I_ext_func=lambda t: 10.0 if 5 <= t <= 6 else 0.0,
)
_ts = res_snap['t']

fig_s, axes_s = plt.subplots(4, 1, figsize=(11, 13), sharex=True)
fig_s.suptitle('Interactive Hodgkin-Huxley Simulator – Static Snapshot\n'
               '(default parameters:  $I_{\mathrm{ext}} = 10 \mu\text{A}/\text{cm}^2$ , pulse 5-6 ms)\n'
               'fontsize=13, fontweight="bold", y=0.99)

ax = axes_s[0]
ax.plot(_ts, res_snap['V'], 'k-', lw=2)
ax.axhline(y=-65, color='gray', ls=':', alpha=.5, label='Resting potential')
ax.axhline(y=-55, color='orange', ls='--', alpha=.5, label='Threshold')
ax.axvspan(5, 6, alpha=.15, color='yellow', label='Stimulus')
ax.set_ylabel('Membrane\nPotential (mV)', fontsize=11)
ax.legend(loc='upper right', fontsize=9)
ax.grid(True, alpha=0.3)

ax = axes_s[1]
ax.plot(_ts, res_snap['m'], 'r-', lw=2, label=r'$m$ (Na+ activation)')
ax.plot(_ts, res_snap['h'], 'r--', lw=2, label=r'$h$ (Na+ inactivation)')
ax.plot(_ts, res_snap['n'], 'b-', lw=2, label=r'$n$ (K+ activation)')
ax.set_ylabel('Gating\nVariable', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.set_ylim(-0.05, 1.05)
ax.grid(True, alpha=0.3)

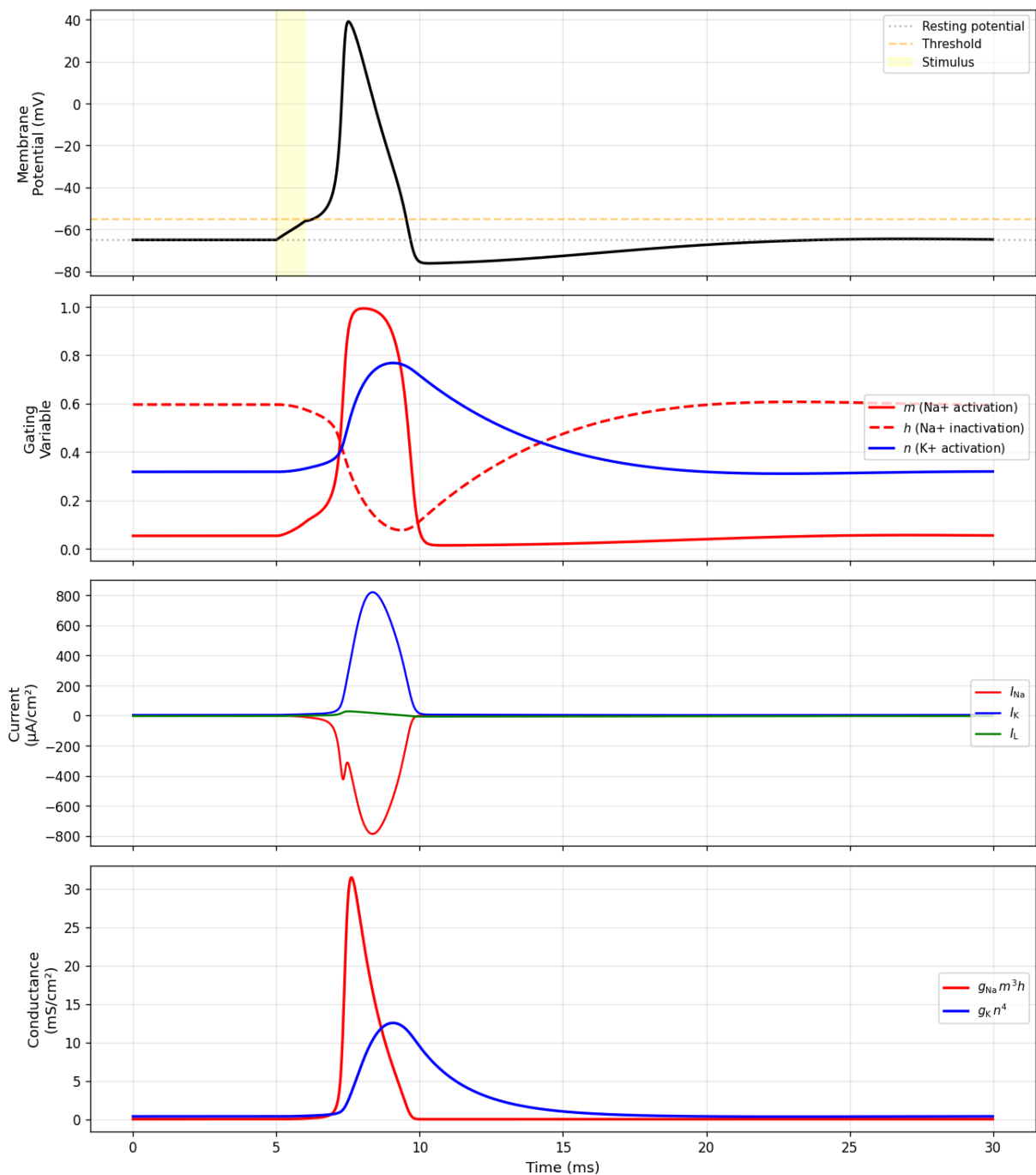
ax = axes_s[2]
ax.plot(_ts, res_snap['I_Na'], 'r-', lw=1.5, label=r'$I_{\mathrm{Na}}$')
ax.plot(_ts, res_snap['I_K'], 'b-', lw=1.5, label=r'$I_{\mathrm{K}}$')
ax.plot(_ts, res_snap['I_L'], 'g-', lw=1.5, label=r'$I_{\mathrm{L}}$')
ax.set_ylabel('Current\n( $\mu\text{A}/\text{cm}^2$ )', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

ax = axes_s[3]
ax.plot(_ts, res_snap['g_Na_t'], 'r-', lw=2, label=r'$g_{\mathrm{Na}}\cdot m^3 h$')
ax.plot(_ts, res_snap['g_K_t'], 'b-', lw=2, label=r'$g_{\mathrm{K}}\cdot n^4$')
ax.set_ylabel('Conductance\n( $\text{mS}/\text{cm}^2$ )', fontsize=11)
ax.set_xlabel('Time (ms)', fontsize=11)
ax.legend(loc='right', fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Interactive Hodgkin-Huxley Simulator — Static Snapshot
(default parameters: $I_{\text{ext}} = 10 \mu\text{A}/\text{cm}^2$, pulse 5–6 ms)



Check Your Understanding: Simulation Results

Q7. Looking at the simulation (Figure 6), why does the Na⁺ conductance (Panel 4, red) have a much *sharper* and *brief*er peak than the K⁺ conductance (blue)?

► Click to reveal answer

Q8. (DIY — Coding Challenge) Modify the simulation to apply a **hyperpolarizing** current pulse ($I_{\text{ext}} = -5 \mu\text{A}/\text{cm}^2$) instead of a depolarizing one. What happens to the

membrane potential? Is an action potential triggered?

- Click for hint
- Click to reveal answer

```
In [39]: # — DIY: Try it yourself! —————
# Uncomment and modify the code below to explore different stimuli.

# Example: Hyperpolarizing pulse
# def I_pulse_hyper(t):
#     return -5.0 if 5 <= t <= 6 else 0.0
#
# results_hyper = hh.simulate(t_max=30, I_ext_func=I_pulse_hyper)
#
# plt.figure(figsize=(10, 4))
# plt.plot(results_hyper['t'], results_hyper['V'], 'k-', lw=2)
# plt.axhline(y=-55, color='orange', linestyle='--', alpha=0.5, label='Thres')
# plt.xlabel('Time (ms)')
# plt.ylabel('V (mV)')
# plt.title('Response to Hyperpolarizing Pulse')
# plt.legend()
# plt.grid(True, alpha=0.3)
# plt.show()
```

8. Repetitive Firing and Frequency–Current Relationship

When a neuron receives a **sustained** (rather than brief) current injection, it can fire **repetitive action potentials**. The firing frequency increases with the strength of the injected current — this is how neurons **encode information** as firing rates.

```
In [40]: # --- Repetitive firing with sustained current ---
hh = HodgkinHuxley()

fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

currents = [7.0, 10.0, 20.0]
colors = ['#2196F3', '#FF9800', '#E91E63']

for i, (I_amp, color) in enumerate(zip(currents, colors)):
    I_func = lambda t, amp=I_amp: amp if 10 <= t <= 80 else 0.0
    results = hh.simulate(t_max=100, I_ext_func=I_func)

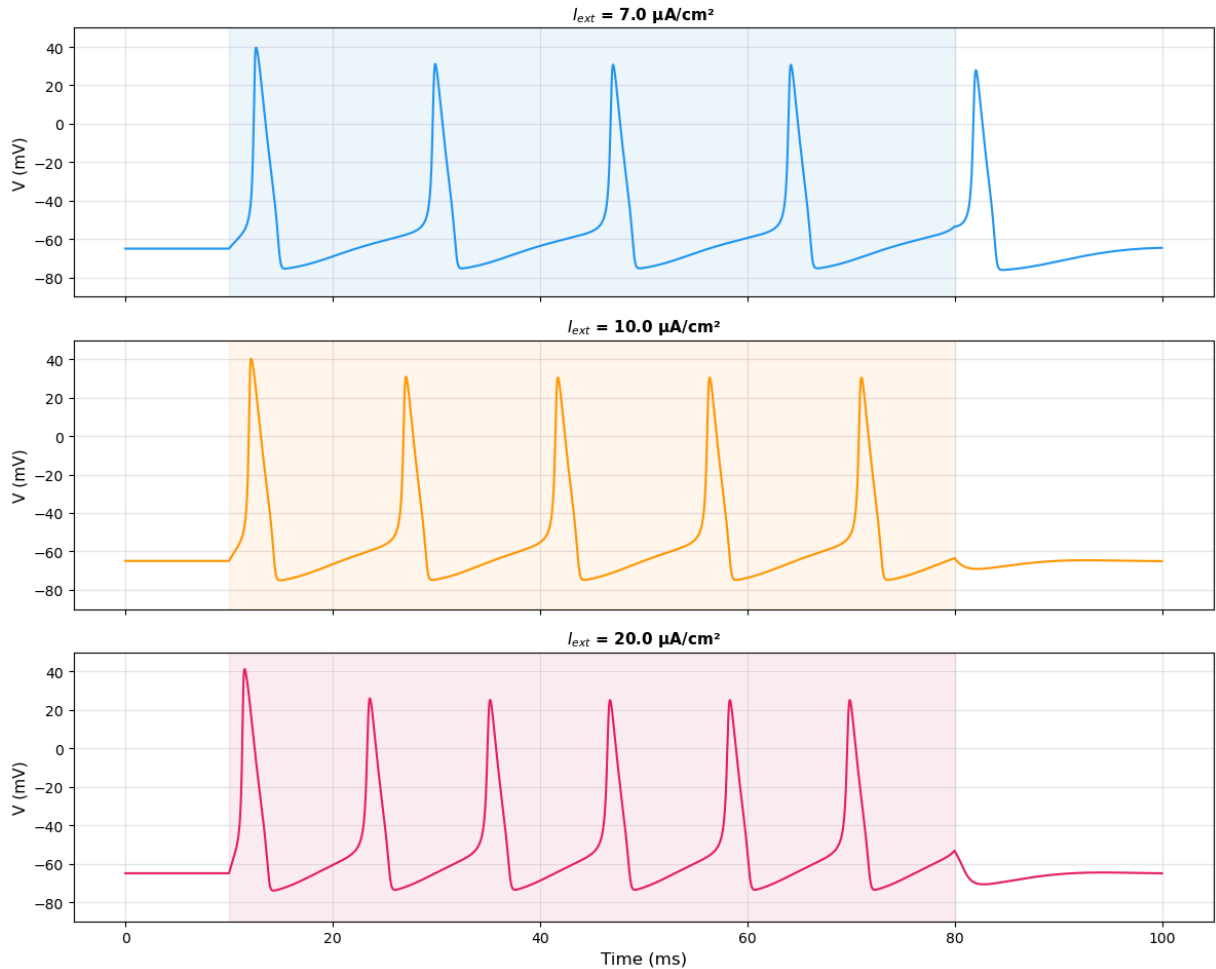
    ax = axes[i]
    ax.plot(results['t'], results['V'], color=color, lw=1.5)
    ax.axvspan(10, 80, alpha=0.08, color=color)
    ax.set_ylabel('V (mV)', fontsize=11)
    ax.set_title(f'$I_{{ext}}$ = {I_amp}  $\mu\text{A}/\text{cm}^2$', fontsize=11, fontweight='b')
    ax.set_ylim(-90, 50)
    ax.grid(True, alpha=0.3)

axes[-1].set_xlabel('Time (ms)', fontsize=12)$ 
```

```
plt.suptitle('Figure 7. Repetitive Firing Under Sustained Current Injection'
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

print("OBSERVATION: As the injected current increases, the neuron fires")
print("more frequently. This frequency coding is fundamental to how")
print("neurons transmit information in the nervous system.")
```

Figure 7. Repetitive Firing Under Sustained Current Injection



OBSERVATION: As the injected current increases, the neuron fires more frequently. This frequency coding is fundamental to how neurons transmit information in the nervous system.

9. Threshold Behavior and the All-or-Nothing Principle

One of the most remarkable features of the action potential is its **all-or-nothing** character: a stimulus either triggers a full-amplitude action potential or fails entirely. There is a sharp **threshold** separating these two outcomes. Let us explore this computationally.

```
In [41]: # --- Threshold behavior: subthreshold vs suprathreshold ---
hh = HodgkinHuxley()
```

```

fig, ax = plt.subplots(figsize=(11, 5))

# Test a range of brief current pulses (1 ms duration)
amplitudes = [2, 4, 5, 6, 7, 10]
cmap = plt.cm.coolwarm
norm = plt.Normalize(vmin=min(amplitudes), vmax=max(amplitudes))

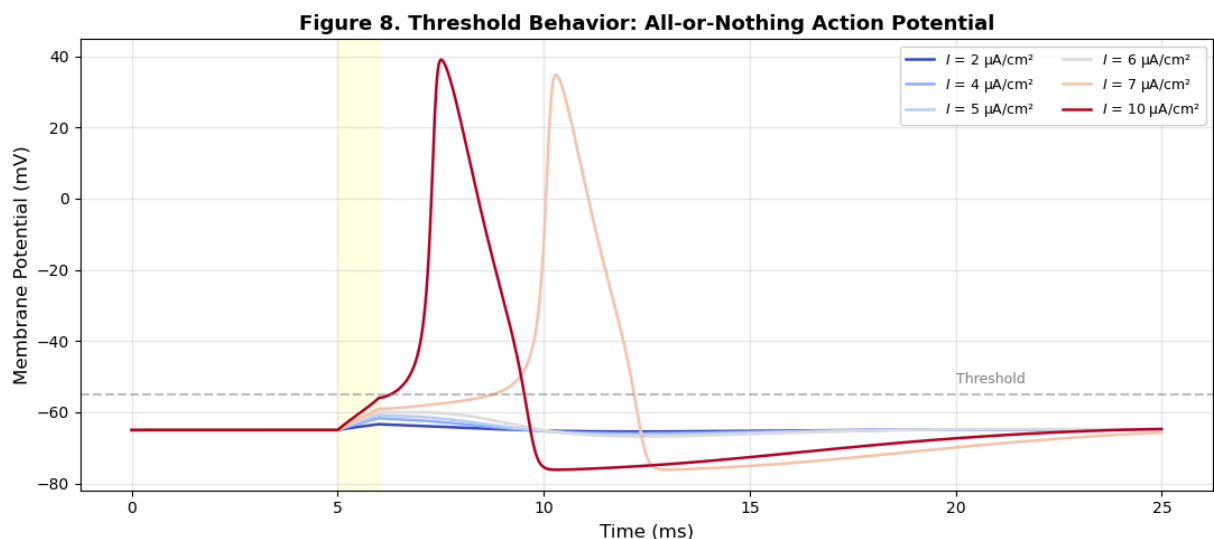
for amp in amplitudes:
    I_func = lambda t, a=amp: a if 5 <= t <= 6 else 0.0
    results = hh.simulate(t_max=25, I_ext_func=I_func)
    color = cmap(norm(amp))
    ax.plot(results['t'], results['V'], color=color, lw=1.8,
            label=f'$I$ = {amp}  $\mu\text{A}/\text{cm}^2$')

ax.axhline(y=-55, color='gray', linestyle='--', alpha=0.5)
ax.text(20, -52, 'Threshold', fontsize=9, color='gray')
ax.axvspan(5, 6, alpha=0.1, color='yellow')

ax.set_xlabel('Time (ms)', fontsize=12)
ax.set_ylabel('Membrane Potential (mV)', fontsize=12)
ax.set_title('Figure 8. Threshold Behavior: All-or-Nothing Action Potential')
ax.legend(loc='upper right', fontsize=9, ncol=2)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("OBSERVATION: Stimuli below threshold (~6  $\mu\text{A}/\text{cm}^2$  for 1 ms) produce")
print("only small, passive responses. Once threshold is crossed, a FULL")
print("action potential is triggered regardless of how much above threshold")
print("the stimulus is. This is the all-or-nothing principle.")$ 
```



OBSERVATION: Stimuli below threshold (~6 $\mu\text{A}/\text{cm}^2$ for 1 ms) produce only small, passive responses. Once threshold is crossed, a FULL action potential is triggered regardless of how much above threshold the stimulus is. This is the all-or-nothing principle.

10. Refractory Periods

After an action potential, the neuron enters a **refractory period** during which it is temporarily unable (or less able) to fire again:

- **Absolute refractory period** (~1-2 ms): No stimulus, however strong, can trigger another AP. Caused by Na⁺ channel inactivation ($h \approx 0$).
- **Relative refractory period** (~3-5 ms): A stronger-than-normal stimulus is required. Caused by lingering K⁺ channel activation (n still elevated) and incomplete recovery of h .

This has important physiological consequences:

- It ensures **unidirectional propagation** of action potentials along the axon
- It sets an **upper limit** on firing frequency (~500-1000 Hz)

Check Your Understanding: Threshold & Refractory Periods

Q9. A neuron is stimulated with two identical depolarizing pulses separated by 1.5 ms. The first pulse triggers a full action potential but the second does not. Why?

► Click to reveal answer

Q10. What is the maximum firing frequency of a neuron if its absolute refractory period is 1 ms and its relative refractory period is an additional 3 ms?

► Click to reveal answer

Q11. Patients with **multiple sclerosis (MS)** experience demyelination of axons. How would you expect this to affect action potential propagation? (*Think about what myelin does.*)

► Click to reveal answer

```
In [42]: # --- Refractory period demonstration ---
hh = HodgkinHuxley()

fig, axes = plt.subplots(3, 1, figsize=(12, 9), sharex=True)
intervals = [5.0, 3.0, 1.5] # ms between two pulses
labels = ['Δt = 5.0 ms (both fire)', 'Δt = 3.0 ms (second reduced)', 'Δt = 1.0 ms (second blocked)']
colors_list = ['#4CAF50', '#FF9800', '#F44336']

for i, (dt_gap, label, col) in enumerate(zip(intervals, labels, colors_list)):
    # Two brief pulses
    t1_start, t1_end = 5.0, 6.0
    t2_start, t2_end = t1_end + dt_gap, t1_end + dt_gap + 1.0

    def I_func(t, s1=t1_start, e1=t1_end, s2=t2_start, e2=t2_end):
        if s1 <= t <= e1 or s2 <= t <= e2:
```



```

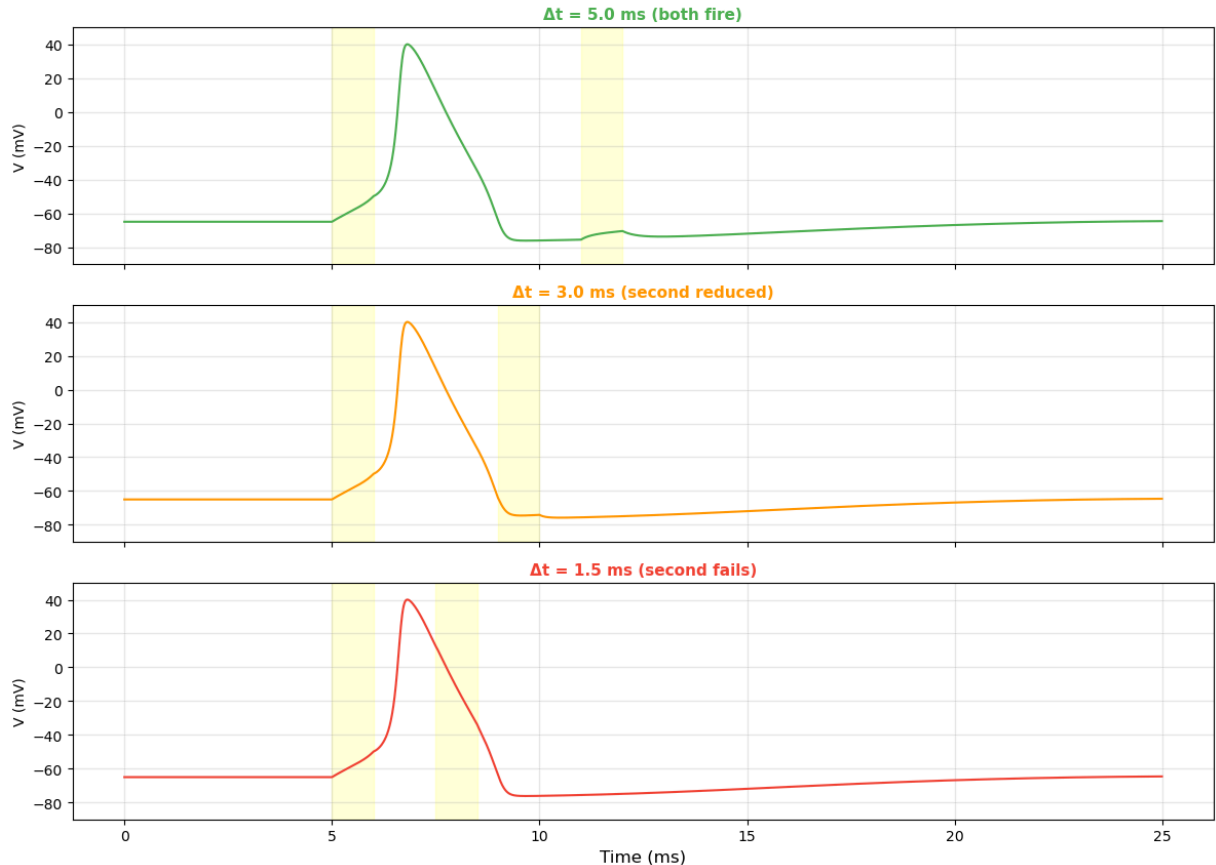
        return 15.0
    return 0.0

results = hh.simulate(t_max=25, I_ext_func=I_func)
ax = axes[i]
ax.plot(results['t'], results['V'], color=col, lw=1.5)
ax.axvspan(t1_start, t1_end, alpha=0.15, color='yellow')
ax.axvspan(t2_start, t2_end, alpha=0.15, color='yellow')
ax.set_ylabel('V (mV)', fontsize=10)
ax.set_title(label, fontsize=11, fontweight='bold', color=col)
ax.set_ylim(-90, 50)
ax.grid(True, alpha=0.3)

axes[-1].set_xlabel('Time (ms)', fontsize=12)
plt.suptitle('Figure 9. Refractory Period: Two Pulses at Different Intervals',
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

```

Figure 9. Refractory Period: Two Pulses at Different Intervals



11. Phase Plane Analysis

A powerful way to understand the dynamics of the HH model is through **phase plane** plots, which show the trajectory of the system in the space of two variables. Below we plot $\dot{V}$ vs the Na^+ conductance ($g_{\text{Na}} m^3 h$) and vs the K^+ conductance ($g_{\text{K}} n^4$) during an action potential. This reveals the characteristic **loop** trajectory of the action potential.

```

In [43]: # --- Phase plane analysis ---
hh = HodgkinHuxley()
results = hh.simulate(t_max=30, I_ext_func=lambda t: 10.0 if 5 <= t <= 6 else 0)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

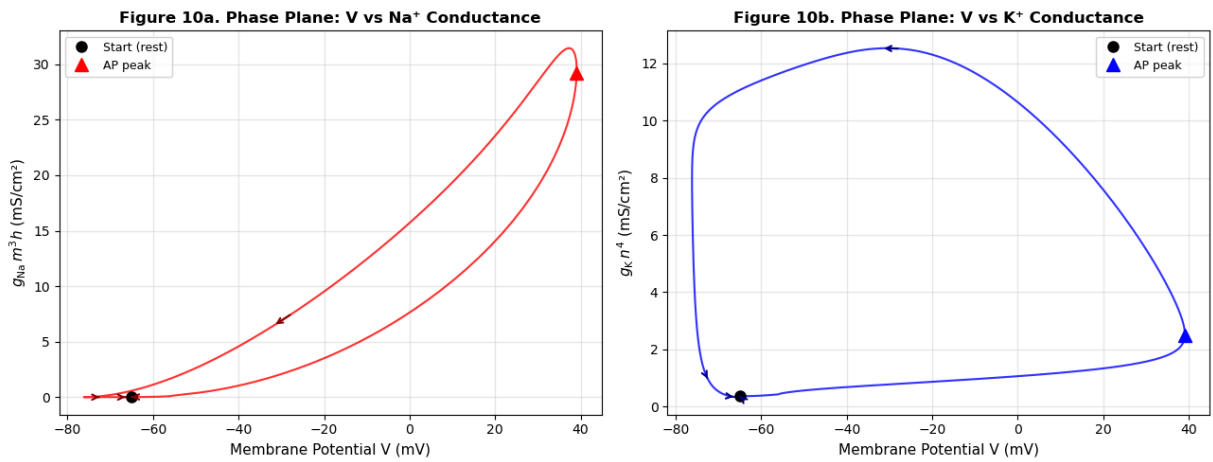
# V vs Na+ conductance
ax = axes[0]
ax.plot(results['V'], results['g_Na_t'], 'r-', lw=1.5, alpha=0.8)
ax.plot(results['V'][0], results['g_Na_t'][0], 'ko', ms=8, label='Start (rest)')
# Mark peak
peak_idx = np.argmax(results['V'])
ax.plot(results['V'][peak_idx], results['g_Na_t'][peak_idx], 'r^', ms=10, label='Peak')
# Add arrows to show direction
for frac in [0.15, 0.3, 0.5, 0.7]:
    idx = int(frac * len(results['t']))
    dx = results['V'][idx+10] - results['V'][idx]
    dy = results['g_Na_t'][idx+10] - results['g_Na_t'][idx]
    ax.annotate('', xy=(results['V'][idx+10], results['g_Na_t'][idx+10]),
                xytext=(results['V'][idx], results['g_Na_t'][idx]),
                arrowprops=dict(arrowstyle='->', color='darkred', lw=1.5))
ax.set_xlabel('Membrane Potential V (mV)', fontsize=11)
ax.set_ylabel(r'$g_{\mathrm{Na}}$ (mS/cm$^2$)', fontsize=11)
ax.set_title('Figure 10a. Phase Plane: V vs Na$^+$ Conductance', fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

# V vs K+ conductance
ax = axes[1]
ax.plot(results['V'], results['g_K_t'], 'b-', lw=1.5, alpha=0.8)
ax.plot(results['V'][0], results['g_K_t'][0], 'ko', ms=8, label='Start (rest)')
ax.plot(results['V'][peak_idx], results['g_K_t'][peak_idx], 'b^', ms=10, label='Peak')
for frac in [0.15, 0.3, 0.5, 0.7]:
    idx = int(frac * len(results['t']))
    ax.annotate('', xy=(results['V'][idx+10], results['g_K_t'][idx+10]),
                xytext=(results['V'][idx], results['g_K_t'][idx]),
                arrowprops=dict(arrowstyle='->', color='darkblue', lw=1.5))
ax.set_xlabel('Membrane Potential V (mV)', fontsize=11)
ax.set_ylabel(r'$g_{\mathrm{K}}$ (mS/cm$^2$)', fontsize=11)
ax.set_title('Figure 10b. Phase Plane: V vs K$^+$ Conductance', fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("The Na$^+$ conductance (left) shows a LOOP: it rises fast during")
print("depolarization and falls as channels inactivate – a TRANSIENT response")
print("The K$^+$ conductance (right) shows a COUNTER-CLOCKWISE loop:")
print("it rises AFTER the voltage peaks and stays elevated during repolarization")

```



The Na⁺ conductance (left) shows a LOOP: it rises fast during depolarization and falls as channels inactivate – a TRANSIENT response. The K⁺ conductance (right) shows a COUNTER-CLOCKWISE loop: it rises AFTER the voltage peaks and stays elevated during repolarization.

DIY Exercises: Explore Further

Exercise 1: Drug Simulation — Channel Blockers

Tetrodotoxin (TTX, from puffer fish) blocks Na⁺ channels. TEA (tetraethylammonium) blocks K⁺ channels.

Simulate the effect of these drugs by reducing the maximum conductances:

- ▶ [Click for starter code](#)
- ▶ [Click to reveal what you should see](#)

Exercise 2: Temperature Dependence

Ion channel kinetics speed up with temperature (approximately $Q_{10} \approx 3$, meaning rates triple for every 10°C increase). The standard HH parameters are for 6.3°C (squid axon in cold seawater).

- ▶ [Click for starter code](#)
- ▶ [Click to reveal expected result](#)

Exercise 3: Frequency–Current (f–I) Curve

A key property of neurons is the relationship between injected current and firing rate. Write code to sweep I_{ext} from 0 to 30 $\mu\text{A}/\text{cm}^2$ and count the number of action potentials in a 100 ms window.

- ▶ [Click for hint](#)
- ▶ [Click to reveal what you should see](#)

```
In [44]: # — DIY: Implement the exercises above —————
# Use this cell to experiment. The HodgkinHuxley class from
# the earlier cell is available.

# --- Exercise 1: Channel blockers ---
# (uncomment and modify)
# hh_ttx = HodgkinHuxley()
# hh_ttx.g_Na = 12.0 # TTX: 10% of normal
# results_ttx = hh_ttx.simulate(t_max=30,
#     I_ext_func=lambda t: 10.0 if 5 <= t <= 6 else 0.0)
# plt.figure(figsize=(10, 4))
# plt.plot(results_ttx['t'], results_ttx['V'], 'r-', lw=2)
# plt.title('Effect of TTX')
# plt.show()

# --- Exercise 2: Temperature ---
# T_ref, T_new, Q10 = 6.3, 37.0, 3.0
# phi = Q10 ** ((T_new - T_ref) / 10.0)
# print(f"φ = {phi:.2f}")

# --- Exercise 3: f-I curve ---
# def count_spikes(V, threshold=-20):
#     return np.sum(np.diff((V > threshold).astype(int)) == 1)
#
# currents = np.arange(0, 31, 1.0)
# spike_counts = []
# for I_amp in currents:
#     res = hh.simulate(t_max=100,
#         I_ext_func=lambda t, a=I_amp: a if 10 <= t <= 90 else 0.0)
#     spike_counts.append(count_spikes(res['V']))
#
# plt.figure(figsize=(8, 5))
# plt.plot(currents, spike_counts, 'ko-', lw=2)
# plt.xlabel('Injected Current (μA/cm²)')
# plt.ylabel('Number of Spikes (in 80 ms)')
# plt.title('f-I Curve')
# plt.grid(True, alpha=0.3)
# plt.show()
```

12. Going Deeper

Extensions of the HH model

The basic HH model can be extended in many directions:

- **Spatial propagation:** Add a diffusion term to simulate action potential propagation along an axon (cable equation)
- **Temperature dependence:** Ion channel kinetics speed up with temperature ($Q_{10} \approx 3$)
- **Synaptic transmission:** Add conductance-based synaptic inputs to model networks

- **Channel blockers:** Simulate the effect of drugs like TTX (tetrodotoxin, blocks Na^+) or TEA (tetraethylammonium, blocks K^+)
- **Stochastic channels:** Replace deterministic gating with Markov chain models to capture channel noise
- **Realistic morphology:** Use compartmental models (e.g., NEURON simulator) to model dendritic trees

Key references

1. **Hodgkin, A.L. & Huxley, A.F.** (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiol.* 117:500–544. [The original paper]
2. **Izhikevich, E.M.** (2007). *Dynamical Systems in Neuroscience: The Geometry of Excitability and Bursting*. MIT Press. [Excellent textbook with geometric intuition]
3. **Dayan, P. & Abbott, L.F.** (2001). *Theoretical Neuroscience*. MIT Press. [Standard computational neuroscience textbook]
4. **Gerstner, W. et al.** *Neuronal Dynamics* — free online textbook:
<https://neurondynamics.epfl.ch/online/Ch2.S2.html>
5. **3Blue1Brown** — Differential equations, a tourist's guide:
https://youtu.be/p_di4Zn4wz4

Useful resources

- APMonitor — Solve Differential Equations:
<https://apmonitor.com/pdc/index.php/Main/SolveDifferentialEquations>
- UiO course notes on HH model:
<https://www.uio.no/studier/emner/matnat/fys/FYS4715/h20/CourseMaterial/Projects/Hod>
- HH model Python gist:
<https://gist.github.com/giuseppebonaccorso/60ce3eb3a829b94abf64ab2b7a56aaef>